



Tutorial Workbook

Stages 0-3 & Stretch Goals

ADI MCU Hackathon

David Brogno • Thomas Harper

June 2026

Contents

Tutorials	3
Follow-Along Tutorial	3
How Each Stage Works	4
Reference Documents	4
Generating PDFs	4
Stage 0: Project Setup	5
The Iterative Approach	5
Environment Setup	7
Design Your ISA	7
Anatomy of a CPU	10
Project Scaffold	12
Running Simulations	15
Set Up the Compiler Backend	16
Copy the Assembler Tooling	16
Remaining YAML Configs	16
Where to Look for Examples	17
What's Next	17
Stage 1: Data Operations	18
ISA: Define Your First Instructions	18
Verilog: Build the Hardware	19
Encoder: Fill In Your Bit Layout	24
Compiler Backend: First Build	27
Test: Compile and Run	30
Debugging	30
Milestone Checklist	31
What's Next	31
Stage 2: Control Flow	32
ISA: New Instructions	32
Verilog: Branch Logic	33
Encoder: Add Branch Support	35
Compiler Backend: Add Branch Support	36
Test: Compile and Run	38
Debugging	38
Milestone Checklist	39
What's Next	39
Stage 3: Function Calls	40
ISA: New Instructions	40
Verilog: Data Memory & Stack	41
Encoder: Add New Opcodes	44
Compiler Backend: Frame Lowering & Call Support	44
Test: The Gold Standard	47
Debugging	48
Milestone Checklist	49
You're Done!	49

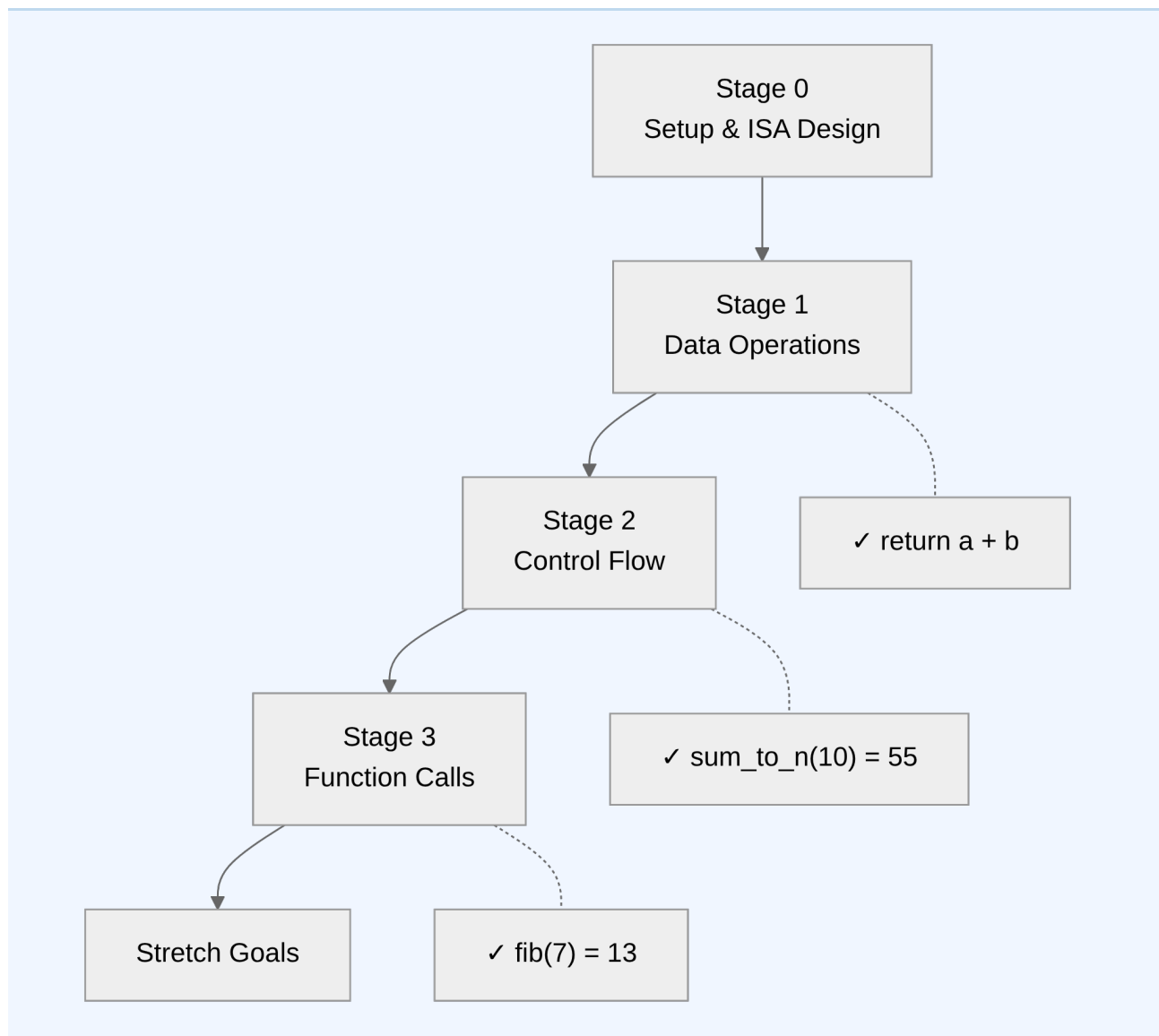
Stage 4: Stretch Goals	52
Additional ALU Operations	52
FPGA Synthesis	52
Performance Tuning	53
Pipelining	53
Peripherals	54
Byte-Addressable Memory	54
Presentation Tips	54
Reference	55
Milestone Checklist	55
Appendix: Glossary	55
A	55
B	55
C	56
D	56
F	56
H	56
I	56
L	57
M	57
O	57
P	57
R	57
S	57
T	58
V	58
W	58

Tutorials

Getting-started guides and workshop materials for ADI MCU Hackathon participants.

Follow-Along Tutorial

This walkthrough contains a series of stages where each is a **full vertical slice** – you design instructions, implement them in Verilog, update the encoder, adapt the compiler backend, compile a C test, and simulate it.



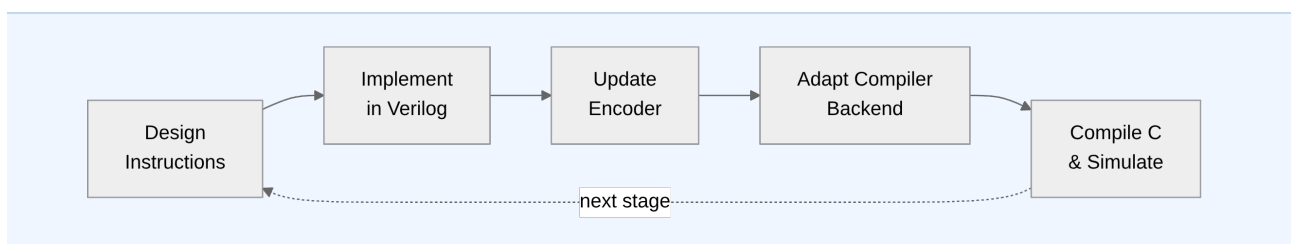
Stage	Title	What you add	Definition of Done
0	Setup & ISA Design	Environment, project skeleton, ISA spec	Project compiles, backend cloned
1	Data Operations	ADD, SUB, MOV, LI	add(21,21) = 42 compiles & runs
2	Control Flow	CMP, JMP, JZ, JNZ, JLT, JGT	sum_to_n(10) = 55 compiles & runs

Stage	Title	What you add	Definition of Done
3	Function Calls	CALL, RET, PUSH, POP, LOAD, STORE	<code>fib(7) = 13</code> compiles & runs
Stretch	Stretch Goals	FPGA, pipelining, peripherals, extra ALU ops	Bonus points

Prerequisites: Basic proficiency in Verilog (modules, always blocks, wire/reg) and C. No prior ISA design, compiler, or FPGA experience assumed.

How Each Stage Works

Every stage follows the same loop:



1. **Design** – decide which instructions to add, document their encoding
2. **Verilog** – implement the hardware, test with hand-encoded hex
3. **Encoder** – add new opcodes to the Python script (fill in bit-packing)
4. **Backend** – add instruction patterns and lowering to the LLVM backend
5. **Test** – compile a C program, simulate, verify the result

Reference Documents

- **Getting Started** – read first: environment setup, tools, and readiness checklist
- **Project Walkthrough** – detailed implementation guide with code templates
- **LLVM Backend Template** – the fill-in-the-blanks backend you start from
- **LLVM Backend Porting Guide** – LLVM concepts and a file-by-file porting walkthrough

Generating PDFs

An ADI-branded PDF build system is included. It produces print-ready documents via LaTeX (XeLaTeX).

Install dependencies (once)

```
sudo apt-get install -y texlive-xetex texlive-latex-extra texlive-fonts-extra pandoc
```

Build all PDFs → output/getting-started.pdf, tutorial-workbook.pdf, project-walkthrough.pdf

```
python3 build_pdf.py
```

Build a specific document

```
python3 build_pdf.py getting-started
```

```
python3 build_pdf.py workbook
```

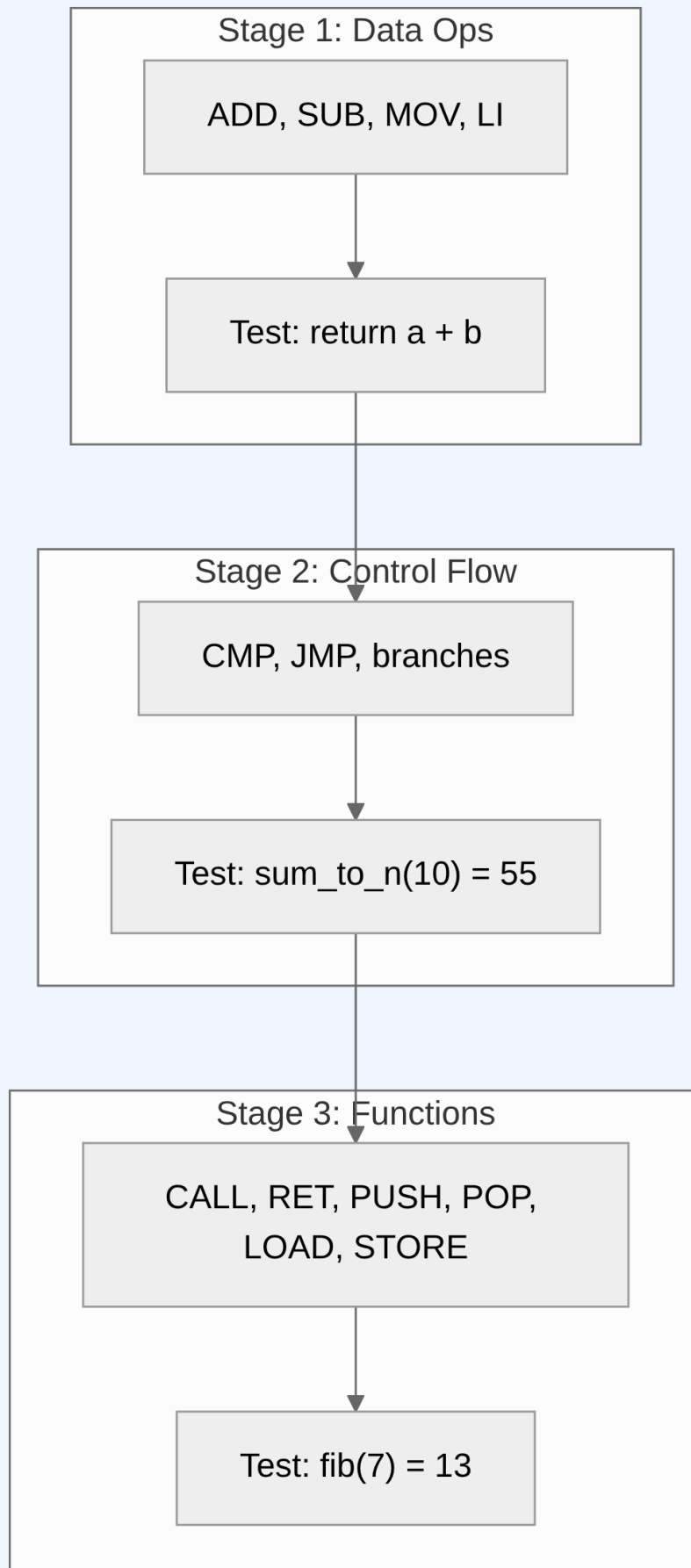
See **latex/README.md** for full details and options.

Stage 0: Project Setup

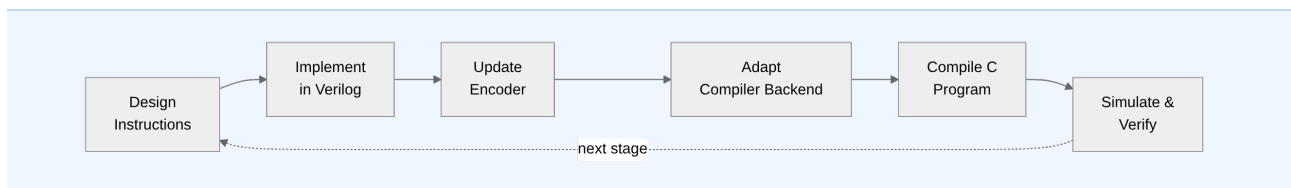
Before building anything, you need to make three sets of decisions, set up your environment, and scaffold the project. This stage produces no running code — but everything that follows depends on getting it right.

The Iterative Approach

You'll build your CPU in three stages. Each stage is a **full vertical slice** — you'll design new instructions, implement them in Verilog, update the hex encoder, adapt the compiler backend, compile a C test program, and simulate it. Each stage ends with a working demo.



Within each stage, the workflow is:



Environment Setup

You should have already been provided the material you need to build a docker container and start working in it. The container includes:

- **LLVM 17** (clang, llc, llvm-link)
- **Icarus Verilog** (iverilog, vvp)
- **GTKWave** (waveform viewer)
- **Yosys** (open-source synthesis)
- **Python 3** with PyYAML
- **CMake + Ninja** (for building the LLVM backend)

Verify Your Tools

Open a terminal inside VS Code and check:

```

iverilog -V
/opt/llvm/bin/llc --version
python3 -c "import yaml; print('OK')"
cmake --version
ninja --version
  
```

All should print version information without errors.

Design Your ISA

Note: The decisions you make here set the widths of every signal in your CPU. Before working through the problems below, skim the [Anatomy of a CPU](#) section – it explains what each module actually needs from the ISA, so these choices will feel concrete rather than abstract.

Before writing any code, sit down with your team and consider what choices you want to make for the following design decisions. You can modify them later if you want but having a strong sense of what you want to achieve early will make it easier as you progress.

Architecture Parameters

Parameter	Trade-off
Data path width	Wider means more expressive instructions and larger immediates, but more hardware everywhere. 16-bit is tight but simple; 32-bit is comfortable.
Register count	More registers means fewer memory accesses, but larger multiplexers and more bits consumed per instruction for register addresses. Powers of two are conventional (8, 16, 32).

Parameter	Trade-off
Instruction width	Wider instructions give you more room for opcode bits, register fields, and immediates. Fixed-width (all instructions the same size) is strongly recommended – it makes decoding much simpler.
Address space	How much memory can your CPU access? This determines how many bits you need in address and branch fields.

Register Roles

Some registers need fixed roles agreed between hardware and software. A strong starting point would be:

Role	What it solves
Zero register	Always reads 0. Makes encoding cleaner (MOV can become ADD rd, r_zero, rs) and lets you discard unwanted results.
Stack pointer	Tracks the top of the call stack. Needed for Stage 3.
Link register	Holds the return address on function call. Needed for Stage 3.
Argument / return registers	Agreed locations for passing function arguments and return values. The compiler and hardware must agree on these.

Example register map: r0=zero, r1=PC, r2=SP, r3=LR, r5=condition result, r8–r15=args/return, r16–r31=callee-saved general purpose.

Instruction Formats

Every instruction is a fixed-width bit pattern. You need to decide how that pattern is laid out. Think of this as a packing problem: you have N bits and need to fit an opcode, source operands, a destination, and possibly an immediate.

Work through these problems and decide an approach for each:

Problem 1: How does the decoder tell instructions apart?

The decoder must know which instruction type it's seeing before it can extract operand fields. – **A dedicated type bit** – You may find it easy to implement a design that has a dedicated bit in the opcode that indicated whether an instruction requires the ALU, or if it is just accessing memory or doing control flow operations. – **The opcode field itself** – Where will you put the opcode in the instruction? Will it be the most significant bits? Least significant? Somewhere in the middle? Split into different bit locations? Unless you are confident, start with a simple design.

Problem 2: How do you get constants into registers?

ALU instructions operate on register values, but programs constantly need literal numbers. – **Inline immediates** – a flag bit in the instruction says “treat this operand field as a literal, not a register address.” Compact, but the value range is limited to however many bits you can spare in that field. – **Dedicated load-immediate instruction** – a separate instruction whose payload is almost entirely the immediate value, giving a much wider range (e.g. 16 bits). Costs an extra instruction to set up, but avoids the range limit. – **Both** – inline immediates for small constants, a load-immediate instruction for larger ones. Most real ISAs do this.

Problem 3: How wide are your immediates and addresses?

Branch targets and memory addresses must be able to reach anywhere in your address space. – If your address space is 16-bit and your instruction is 32-bit, you have room for a full 16-bit address alongside an opcode and a register. – If your instruction is 16-bit, something has to give – consider PC-relative branches with a smaller offset, or a two-instruction sequence to load large addresses.

Problem 4: Do the bits add up?

Once you've made the above decisions, write out the exact bit-field layout for each instruction format you want to use (might be 1 format, might be many). Verify the fields fit within your instruction width with no gaps or overlaps. For each field: name, bit positions, and meaning.

Example encoding: Type1(ALU): [31]=1 | [30:17]=opcode(14) | [16]=arg1_is_imm | [15:11]=arg1 | [10]=arg2_is_imm | [9:5]=arg2 | [4:0]=dest Type2(Mem/Control): [31]=0 | [30:22]=opcode(9) | [21]=reg_or_imm | [20:16]=register | [15:0]=address/immediate

Verifying your layout: Write every field and its width in a table, then sum them. The total must equal your instruction width. Here is an example with deliberately different numbers – not the encoding above – to show the method:

Field	Width	Reason
Type bit	1	fixed
Opcode	6	allows 64 distinct instructions
Immediate flag A	1	fixed
Operand A	4	$\log_2(16) = 4$
Immediate flag B	1	fixed
Operand B	4	$\log_2(16) = 4$
Destination	4	$\log_2(16) = 4$
Padding	11	unused
Total	32	✓

Memory Layout**Problem 5: Shared or separate instruction and data memory?**

- **Harvard architecture** – two independent memories, one for instructions and one for data. The CPU can fetch an instruction and read/write data in the same clock cycle. Two simple reg arrays in Verilog. This is the right choice for a single-cycle design.
- **Von Neumann architecture** – one unified memory. Programs can modify themselves and data can be code, but you can't fetch and access data simultaneously in a single cycle without pipelining.

Problem 6: Word-addressed or byte-addressed?

- **Word-addressed** – address 0 is the first instruction word, address 1 is the second. No alignment logic needed. The downside: you can't address individual bytes, which matters when compiling C code that uses char or uint8_t.
- **Byte-addressed** – address 0 is byte 0, address 4 is the second 32-bit word. What C compilers expect. Requires stripping low-order address bits when indexing a word-wide memory array.

Instruction memory is almost always word-addressed regardless of your choice for data memory. If you plan to compile C, byte-addressing for data memory is the expected default.

Problem 7: Where does the stack live?

In most modern ISAs, the stack grows downward from a high address. Decide: - What is the initial stack pointer value? (Top of data memory is conventional.) - How much room does the stack have before it collides with heap or globals?

The compiler's startup stub will initialise SP to whatever address you specify.

Problem 8 (more advanced): How do you expose hardware registers?

Any commercial CPU will have some registers that perform special functions or interact with other pieces of digital hardware. For example, it is common to have a register act as a simple cycle counter to track time, or any sort of off-chip IO will need to interface with the core one way or another. The overwhelming majority of systems do this with **memory-mapped registers**

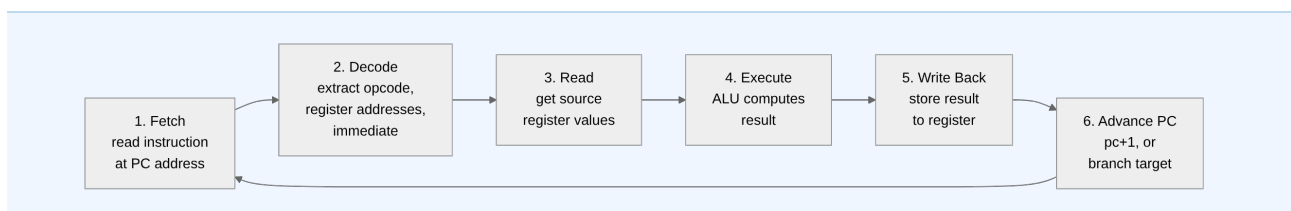
For example, reserve addresses with the MSB set (e.g. 0x8000 and above) for hardware registers rather than RAM. A load from 0x8000 reads the cycle counter; your memory bus detects the high address bit and routes the access accordingly. You don't need to implement this now, but if you think you will want to do something like this then it is good to consider the implications now.

Anatomy of a CPU

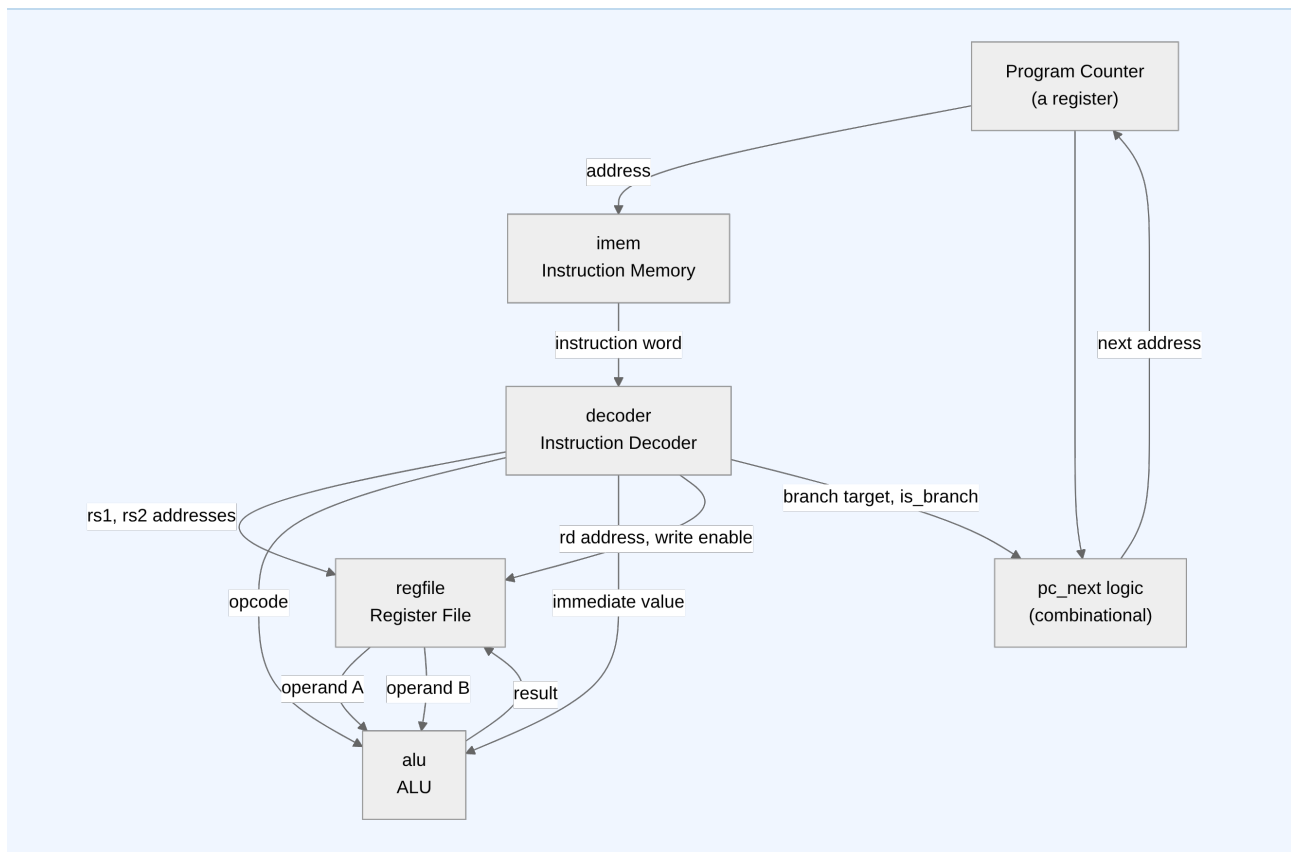
Before you write any Verilog, it's worth understanding what the modules you're about to build actually do and how they're connected. A CPU is not a bag of independent components – it's a single data path. The widths of every signal flowing through it are determined by the architecture decisions you just made.

The Execution Cycle

Every clock cycle, your CPU does the same thing:



These six steps map directly to your modules:



What Each Module Does

Program Counter (PC) – a single register holding the address of the instruction currently being executed. It resets to your reset vector (usually 0x0000) and increments by 1 each cycle unless a branch redirects it. It's not a module of its own – just a reg in your top-level CPU.

Instruction Memory (imem) – a read-only array of instruction words, initialised from a hex file. The address input is the PC; the output is the instruction word for that address. Because it's read-only and word-addressed by the PC, this is the simplest memory you'll ever write.

Decoder (decoder) – purely combinational. Takes the raw instruction word and extracts every field: opcode, source register addresses, destination register address, and any immediate value embedded in the encoding. Also generates control signals (reg_write_en, is_branch, etc.) that tell the rest of the CPU what to do this cycle. This is the most ISA-specific module you'll write – it directly implements your bit-field layout.

Register File (regfile) – a small, fast array of registers. Two asynchronous read ports (so you can read two source operands in the same cycle, combinationally) and one synchronous write port (the result is written on the clock edge). If your ISA has a zero register, the read logic enforces it: reads from that address always return 0, regardless of what was written.

ALU (alu) – purely combinational. Takes two operands and an opcode, outputs the result. In Stage 1 this is just ADD, SUB, and MOV; you add more operations in later stages.

Why Everything Is Tied Together

These modules share widths. When you write `reg [DATA_MSB:0] regs [0:REG_COUNT-1]` in the register file, that `DATA_MSB` comes from `DATA_WIDTH` in your config. When the decoder extracts a 5-bit register address from the instruction word and feeds it to the register file, both modules must agree on that width – and both derive it from `REG_ADDR_W` in your config.

This is why you define all your architecture parameters in a single `config.vh` before writing any module.

A change to REG_COUNT flows automatically to the register file, the decoder, the ALU operand width, and every signal in the CPU without you hunting down every hard-coded number.

Project Scaffold

Directory Structure

```
my_cpu/
├── rtl/
│   ├── include/
│   │   ├── config.vh          # Architecture parameters – fill these in now
│   │   └── opcodes.vh        # Opcode defines – grows each stage
│   ├── cpu.v                 # Top-level module (wires everything together)
│   ├── imem.v               # Instruction memory
│   ├── decoder.v            # Instruction decoder
│   ├── regfile.v            # Register file
│   ├── alu.v                # ALU
│   ├── branch.v             # Branch / PC-next logic (added in Stage 2)
│   ├── dmem.v               # Data memory (added in Stage 3)
│   └── tb.v                 # Testbench
├── tools/
│   ├── isa_config.py         # YOUR opcode tables + encoding functions
│   └── asm.py                # Generic assembler logic (label resolution, two-pass)
├── tests/                    # Test programs (.hex and .c files)
├── llvm-backend/             # Compiler backend (from the template)
├── build/                    # Generated outputs (add to .gitignore)
├── my_cpu.build-compiler.yml
├── my_cpu.compile.yml
└── my_cpu.simulate.yml
```

Configuration Header

This is the foundation everything else is derived from. Fill in your architecture decisions before writing any module:

```
// rtl/include/config.vh
`ifndef MY_CONFIG_VH
`define MY_CONFIG_VH

// Fill these in from your ISA design decisions
`define DATA_WIDTH    ??    // register and ALU width in bits (e.g. 32)
`define ADDR_WIDTH     ??    // address bus width in bits (e.g. 16)
`define INSTR_WIDTH    ??    // instruction word width in bits (e.g. 32)
`define REG_COUNT      ??    // number of registers (e.g. 32)
`define REG_ADDR_W     ??    // bits needed to address a register = clog2(REG_COUNT)
`define IMEM_DEPTH     ??    // instruction memory size in words (e.g. 8192)
`define DMEM_DEPTH     ??    // data memory size in words or bytes (e.g. 32768)

// Derived constants – leave these as-is
`define DATA_MSB      (`DATA_WIDTH - 1)
`define ADDR_MSB       (`ADDR_WIDTH - 1)
`define PC_INIT        {`ADDR_WIDTH{??}} // reset vector: what address should the
first instruction be fetched from?
```

```
`endif
```

Every module includes this header. When you choose 32 registers (`REG_COUNT = 32`), the register file's array, the decoder's register address fields, and the register address inputs to the ALU all automatically use `REG_ADDR_W = 5`. You only ever change one number.

Testbench

Now is a good time to write a test bench that you can use as your harness to test your design throughout this whole project. It's generic enough to work across all three stages — as you add more modules (decoder, regfile, ALU), they slot in under the same `cpu` top-level module and the testbench doesn't change.

Warning: Naming Contracts — If you want to use the provided tooling, the testbench and `simulate.py` are written to reach inside your design by hierarchical path. This is done using fixed names, so getting them wrong causes either a compilation error or silent incorrect behaviour with no diagnostic.

Required name	Must appear in	If wrong
<code>u_imem</code>	Instruction memory instance in <code>cpu.v</code>	Program never loaded; CPU likely executes only NOPs
<code>mem</code>	The actual storage inside instruction memory (<code>imem.v</code>)	Same as above
<code>u_regfile</code>	Register file instance in <code>cpu.v</code>	Register dump fails; <i>verify</i> : never passes
<code>regs</code>	Array of actual registers inside <code>regfile.v</code>	Same as above
<code>dbg_halt</code>	Output port on your top-level <code>cpu</code> module	Simulation runs until <code>MAX_CYCLES</code> timeout regardless of how long the program took

`dbg_halt` must be an actual output port declared on your `cpu` module. Assert it high when your HALT instruction is executing. Until you implement HALT, tie it to `1'b0` — the simulation will time out rather than hang indefinitely.

`simulate.py` compiles the Verilog files listed in your `rtl.files` config. If a file isn't listed, it won't be compiled. If a listed file has syntax errors, the entire simulation will fail to build.

```
// rtl/tb.v
`include "config.vh"
`timescale 1ns / 1ps

module test_harness;
    reg clk    = 0;
    reg rst_n = 0;

    wire [`ADDR_MSB:0] dbg_pc;
    wire                dbg_halt;

    always #5 clk = ~clk; // 100 MHz clock
```

```

cpu u_cpu (
    .clk(clk),
    .rst_n(rst_n),
    .dbg_pc(dbg_pc),
    .dbg_halt(dbg_halt)
);

reg [255*8-1:0] program_file;
integer        max_cycles;
integer        cycle_count;

initial begin
    // Program file and cycle limit come from command-line plusargs,
    // set automatically by simulate.py from your .yaml config.
    if (!$value$plusargs("PROGRAM=%s", program_file)) begin
        $display("ERROR: no +PROGRAM=<file> specified");
        $finish;
    end
    if (!$value$plusargs("MAX_CYCLES=%d", max_cycles))
        max_cycles = 100000;

    $readmemh(program_file, u_cpu.u_imem.mem);
    $dumpfile("cpu_tb.vcd");
    $dumpvars(0, test_harness);

    #25 rst_n = 1; // hold reset for a few cycles

    cycle_count = 0;
    while (!dbg_halt && cycle_count < max_cycles) begin
        @(posedge clk);
        cycle_count = cycle_count + 1;
    end

    if (dbg_halt)
        $display("CPU halted after %0d cycles", cycle_count);
    else
        $display("TIMEOUT after %0d cycles (no HALT reached)", cycle_count);

    // Print register state on completion.
    // simulate.py parses this output to verify results.
    // Format must be: rN = 0x<hex>
    begin : dump_regs
        integer i;
        for (i = 0; i < `REG_COUNT; i = i + 1)
            $display("r%0d = 0x%08h", i, u_cpu.u_regfile.reg[i]);
    end

    $finish;
end
endmodule

```

The register dump format(`r0 = 0x00000000`) is exact — `simulate.py` uses a pattern match on each line. Different spacing or casing means `verify`: checks silently never pass.

Running Simulations

To run a simulation, all you need to do is set up your YAML config and pass it to the `simulate` script. You can write your own tooling if you'd prefer, but it will be faster to use the provided infrastructure.

```
python3 scripts/simulate.py my_cpu.simulate.yml
```

The script handles iverilog compilation, vvp simulation, result parsing, and basic pass/fail reporting — all from the config file.

Simulation Config

```
# my_cpu.simulate.yml
simulate:
  output-dir: ./build/sim

  # Point at a hand-crafted hex file (Stages 1-2) or compiled output (Stage 3+)
  program: ./tests/test.hex

  rtl:
    base-dir: ./rtl
    files:
      - my_pkg.v
      - my_cpu.v
      - my_tb.v
    # Files are resolved relative to base-dir (order does not matter).

    # Optional: verify specific register values after the CPU halts.
    # Leave empty or comment out while hardware is still being built.
    verify: []

  options:
    max-cycles: 10000
    trace: false # set true to print every instruction as it executes
```

As your project grows, update `program`: to point at different test files. When you're ready to verify results, add entries to `verify`:

```
verify:
  - register: r8
    value: 42
```

Viewing Waveforms

After running a simulation, open the VCD file directly:

```
gtkwave build/sim/cpu_tb.vcd
```

`trace: true` in the YAML config enables a cycle-by-cycle PC and instruction trace printed to the terminal during simulation. It is compiled in via an `\ifdef TRACE` block in your testbench. To use it, add this inside the simulation loop in `yourtb.v` (after the `@(posedge clk)` line):


```
`ifdef TRACE
    $display("[%6d] PC=0x%04h INSTR=0x%08h", cycle_count, dbg_pc, u_cpu.instr);
`endif
```

Replace `u_cpu.instr` with the hierarchical path to the instruction signal in your design. Without this block, `trace: true` still compiles with `-DTRACE` but nothing extra is printed.

\$readmemh warning: Every simulation will print a line like *Not enough words in the file for the requested range [0:8191]. This is harmless 2014 your program is shorter than instruction memory. The CPU never reaches the uninitialised locations because HALT stops the simulation first. Ignore it every time.*

Use `simulate.py -v` to see the exact `iverilog` and `vvp` commands being run 2014 useful when compilation fails and you want to run the command manually to inspect the full error output.

Set Up the Compiler Backend

Start from the fill-in-the-blanks backend template. Copy it into your project and give it your ISA's name:

```
cp -r resources/software/template/llvm-backend my_cpu/llvm-backend
```

The template uses the placeholder name `MYISA` (files named `MYISA*`, `-march=myisa`). All the LLVM plumbing compiles as-is; the ISA-specific files are marked with `// TODO`: for you to complete across the three stages. See `resources/software/template/README.md` for optional renaming.

Name consistency: The target name (*myisa* by default, or whatever you rename it to) must match the target: field in `my_cpu.compile.yml` exactly. Pick a name now and keep it consistent across both files.

Copy the Assembler Tooling

A ready-to-use Python assembler ships in `resources/software/template/tools/`. It is split into two files so the ISA-specific part is isolated:

- **asm.py** — Generic assembler engine: label resolution, two-pass structure, argument parsing. Copy it as-is; you should not need to edit it.
- **isa_config.py** — ISA-specific: opcode tables, register map, bit-field layouts, and the `encode_*`() functions. This is the file you rewrite for your own ISA.

Copy both into your project:

```
cp resources/software/template/tools/asm.py          my_cpu/tools/asm.py
cp resources/software/template/tools/isa_config.py    my_cpu/tools/isa_config.py
```

`isa_config.py` ships with a working **example** encoding so the toolchain runs immediately. In Stage 1 you'll replace those opcode tables and layouts with your own design; `asm.py` stays untouched because it imports everything ISA-specific from `isa_config.py`.

Remaining YAML Configs

my_cpu.build-compiler.yml — used to build your LLVM backend:

```
build-compiler:
  llvm:
```

```

src-dir: ""           # defaults to /llvm-src inside the container
build-dir: ""         # defaults to /llvm-build
install-dir: ""       # defaults to /opt/llvm
jobs: 0               # 0 = use all CPU cores
verify: true          # check llc --version lists your target after build
target:
  backend-dir: ./llvm-backend
  name: ""             # auto-detected from CMakeLists.txt

```

my_cpu.compile.yml – used to compile C programs to hex:

```

compile:
  output-dir: ./build/compile
  compiler:
    install-dir: /opt/llvm
    target: myisa          # must match the name registered in your backend
    misc: [-O1]
    codegen: [-O0]
  output:
    asm: true              # keep the .asm intermediate for inspection
    hex: true
    name: program
    assembler: ./tools/asm.py
  startup:
    entry: main
    args: []               # values loaded into argument registers before calling
main
  groups:
    - group: Sources
      files:
        - file: ./tests/test.c

```

Where to Look for Examples

Each stage of the **Project Walkthrough** includes a worked **example** (config header, decoder, testbench, encoder, and assembler) showing one complete set of choices. Read the relevant example, understand the approach, then implement your own version for your own ISA.

What's Next

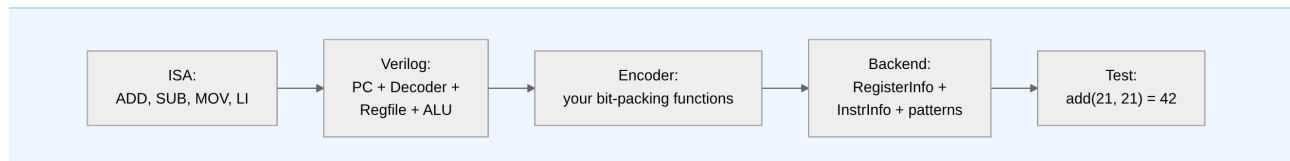
- **Stage 1: Data Operations** – build the data path, implement ADD/SUB/MOV, run your first C function

Stage 1: Data Operations

Goal: Compile `int add(int a, int b) { return a + b; }` in C and run it on your CPU in simulation.

This is your first full vertical slice. By the end, you'll have a CPU that executes arithmetic instructions, a compiler backend that targets it, and a working end-to-end pipeline from C source to simulated hardware.

Before writing any Verilog, sit down with your team and make your ISA design decisions: word size, register count, instruction encoding, opcode assignments. Write these down — this is your ISA spec and you'll refer to it constantly. Fill in the `config.vh` placeholders from Stage 0 with your chosen values.



ISA: Define Your First Instructions

You need just enough to compute with registers and load constants into them. At minimum:

Instruction	What it does	Why you need it
ADD rd, rs1, rs2/imm	rd = rs1 + rs2	Basic arithmetic
SUB rd, rs1, rs2/imm	rd = rs1 - rs2	Subtraction
MOV rd, rs1	rd = rs1	Register copy (the compiler needs this constantly)
LI rd, #imm	rd = immediate	Load a constant into a register

You also need **HALT** so the testbench knows when the program finishes. The simplest implementation is a jump-to-self pseudo-instruction.

Record Your Encoding

For each instruction you've added, write the complete bit-field layout in your ISA spec document. Then add a define for each opcode in `opcodes.vh`, and add the opcode field width to your `config.vh`:

```
// rtl/include/opcodes.vh
`ifndef OPCODES_VH
`define OPCODES_VH

// Fill in YOUR opcode values from your ISA spec
`define OP_ADD    ??
`define OP_SUB    ??
`define OP_MOV    ??
`define OP_LI     ??

`endif
```

Also add the opcode field width to `config.vh` so your ALU can declare its `op` port without a hard-coded number:

```
// In config.vh – add alongside your other parameters
// If your ISA has two instruction types with different opcode sizes,
// add one define per type (e.g. T1_OPCODE_W and T2_OPCODE_W).
`define OPCODE_W ?? // bits in your opcode field
```

Verilog: Build the Hardware

Work through the modules in this order. Each one is independently testable before moving to the next.

Naming Contracts – Your testbench and *simulate.py* reach inside the design by hierarchical path. If you haven't already, review the naming contracts table in Stage 0's Testbench section. The critical names are: *u_imem*, *mem*, *u_regfile*, *regs*, and *dbg_halt*.

Instruction Memory

We use a Harvard architecture: separate instruction and data memory paths. Instruction memory is a simple, asynchronous, purely combinational read from an initialised reg array. It is **word-addressed** – each address points to one instruction word, not one byte.

```
// rtl/imem.v
`include "config.vh"

module imem #(
    parameter DEPTH = `IMEM_DEPTH
)(
    input  wire [`ADDR_MSB:0]    addr,
    output wire [`INSTR_WIDTH-1:0] data
);
    reg [`INSTR_WIDTH-1:0] mem [0:DEPTH-1];
    assign data = mem[addr];
endmodule
```

No changes needed here – it's fully generic. The testbench loads the program into mem via \$readmemh using a hierarchical reference, so the storage array must be named mem.

Program Counter

In your top-level CPU, a register that resets to your reset vector and increments each cycle:

```
reg [`ADDR_MSB:0] pc;
wire [`ADDR_MSB:0] pc_next;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) pc <= `PC_INIT;
    else      pc <= pc_next;
end

// imem is word-addressed: +1 means next instruction, not next byte
assign pc_next = pc + 1; // Stage 2 replaces this with branch logic
```

Because instruction memory is word-addressed, incrementing the PC by 1 fetches the next instruction. Keep this in mind when you add branch offsets in Stage 2 – offsets are in words, not bytes.

Decoder

The decoder is the most ISA-specific module you'll write. It takes the raw instruction word and extracts every field, producing named signals that the rest of the CPU uses.

Start from your ISA spec. For each field in each instruction format, write one assign that slices out the right bits. The port list below is a starting point — add or remove outputs to match what your CPU actually needs:

```
// rtl/decoder.v
`include "config.vh"
`include "opcodes.vh"

module decoder (
    input wire [`INSTR_WIDTH-1:0] instr,

    // Outputs: one per field in your encoding
    // These names and widths should match YOUR ISA spec
    output wire      is_alu_op,      // is this an arithmetic/logic instruction?
    output wire [??:0] opcode,      // which operation?
    output wire [??:0] rd,          // destination register
    output wire [??:0] rs1,          // source register 1
    output wire [??:0] rs2,          // source register 2 (or immediate)
    output wire [??:0] immediate,    // immediate value
    output wire      imm_select,     // use immediate instead of rs2?
    output wire      reg_write_en    // should we write back a result?
    // Add more outputs as your design requires
);

    // Extract fields from the instruction word.
    // The bit ranges must match the layout you documented in your ISA spec.
    assign opcode    = instr[??:??];
    assign rd        = instr[??:??];
    assign rs1       = instr[??:??];
    // ... fill in the rest ...

    // Control signals derived from the opcode
    assign is_alu_op  = /* true when opcode is an arithmetic/logic op */;
    assign reg_write_en = /* true for instructions that write to a register */;

endmodule
```

Tip: Once you've written your decoder, test it in isolation before connecting the rest. Hand-encode a single ADD instruction from your spec, put it in a one-line hex file, and run the simulation with *trace: true*. You should see the correct PC and instruction word on the first clock cycle. If any decoded field shows x in GTKWave, fix the bit slices before moving on.

Register File

A synchronous-write, asynchronous-read register file. Writes happen on the clock edge; reads are combinational. If your ISA has a zero register, enforce it here. The storage must be named *regs* for the testbench register dump to work.

```
// rtl/regfile.v
`include "config.vh"

module regfile (
    input wire      clk,
    input wire
```

```

input wire [`REG_ADDR_W-1:0] wr_addr,
input wire [`DATA_MSB:0] wr_data,
input wire [`REG_ADDR_W-1:0] rd_addr_a,
input wire [`REG_ADDR_W-1:0] rd_addr_b,
output wire [`DATA_MSB:0] rd_data_a,
output wire [`DATA_MSB:0] rd_data_b
);
reg [`DATA_MSB:0] regs [0:`REG_COUNT-1];

// Synchronous write – adapt the zero-register guard to your design.
// Replace 0 below with whichever register number you chose as your zero reg.
always @(posedge clk)
    if (wr_en && wr_addr != 0) // 0 = zero register – change to your choice
        regs[wr_addr] <= wr_data;

// Asynchronous reads – enforces the zero register on the read side too.
// If your ISA has no zero register, just write: assign rd_data_a =
regs[rd_addr_a];
assign rd_data_a = (rd_addr_a == 0) ? `{DATA_WIDTH{1'b0}} : regs[rd_addr_a];
assign rd_data_b = (rd_addr_b == 0) ? `{DATA_WIDTH{1'b0}} : regs[rd_addr_b];
endmodule

```

Replace both 0 literals with the register number you designated as the zero register.

Tip: Test the zero-register guard early. Hand-encode instructions that write 99 to a normal register and 0 to your zero register. The zero register should still read 0x00000000 in the register dump regardless of the write attempt.

ALU

A purely combinational function of its inputs. The opcode selects the operation. Add one case entry per instruction:

```

// rtl/alu.v
`include "config.vh"
`include "opcodes.vh"

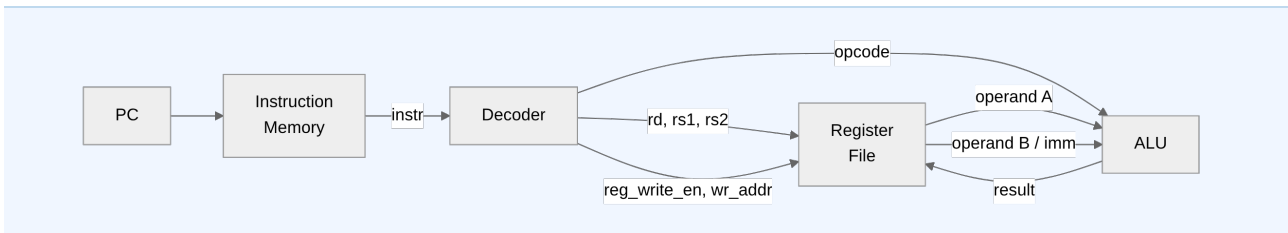
module alu (
    input wire [`OPCODE_W-1:0] op, // use a config define for opcode width
    input wire [`DATA_MSB:0] a,
    input wire [`DATA_MSB:0] b,
    output reg [`DATA_MSB:0] result
);
always @(*) begin
    case (op)
        `OP_ADD: result = a + b;
        `OP_SUB: result = a - b;
        `OP_MOV: result = a;
        // Add more operations in later stages
        default: result = `{DATA_WIDTH{1'b0}};
    endcase
end
endmodule

```

Add `OPCODE_W` to your `config.vh` — it's the width of your opcode field, derived from your encoding.

Top-Level CPU

The top-level module wires everything together. The key data paths are:



Build it up section by section. Here's the skeleton — fill in the wiring based on your decoder's actual output signals:

```

// rtl/cpu.v
`include "config.vh"

module cpu #(
    parameter IMEM_INIT_FILE = ""
) (
    input wire          clk,
    input wire          rst_n,
    output wire [`ADDR_MSB:0] dbg_pc,
    output wire          dbg_halt
);

// — Program Counter —————
reg [`ADDR_MSB:0] pc;
wire [`ADDR_MSB:0] pc_next;
assign dbg_pc = pc;
assign pc_next = pc + 1; // Replaced in Stage 2

always @(posedge clk or negedge rst_n)
    if (!rst_n) pc <= `PC_INIT;
    else      pc <= pc_next;

// — Instruction Fetch —————
wire [`INSTR_WIDTH-1:0] instr;
imem #(.DEPTH(`IMEM_DEPTH)) u_imem (.addr(pc), .data(instr));

// — Decode —————
// Declare wires matching your decoder's outputs, then instantiate.
// (Fill in based on your decoder's port list)
wire          is_alu_op;
wire [??:0] opcode;
wire [??:0] rd, rs1, rs2;
wire [??:0] immediate;
wire          imm_select;
wire          reg_write_en;

decoder u_decoder (
    .instr(instr),

```

```

    // ... connect each port ...
);

// — Register File —————
wire [`DATA_MSB:0] rf_a, rf_b;
wire [`DATA_MSB:0] wb_data;

regfile u_regfile (
    .clk(clk),
    .wr_en(reg_write_en),
    .wr_addr(rd),
    .wr_data(wb_data),
    .rd_addr_a(rs1), .rd_data_a(rf_a),
    .rd_addr_b(rs2), .rd_data_b(rf_b)
);

// — Operand mux (register vs immediate) —————
wire [`DATA_MSB:0] alu_a = rf_a;
wire [`DATA_MSB:0] alu_b = imm_select ? /* zero-extend immediate */ : rf_b;

// — ALU —————
wire [`DATA_MSB:0] alu_result;
alu u_alu (.op(opcode), .a(alu_a), .b(alu_b), .result(alu_result));

// — Write-back mux —————
// Where does wb_data come from?
// Option A (simpler): treat LI as "ADD rd, r0, #imm" — it goes through
// the ALU just like any other instruction, so wb_data = alu_result always.
// This works if your ALU correctly handles immediates and your zero
// register always reads 0.
// Option B (separate path): LI has its own write-back mux entry:
// is_alu_op → alu_result
// is_li     → zero-extended immediate
// Pick one, be consistent in your decoder and your ALU.
// (Stage 3 adds: is_load → memory read data)
assign wb_data = /* your mux here */;

// — Halt detection —————
// Halt when the CPU executes a jump to its own address.
// Add this after Stage 2 adds branch logic; for now a placeholder:
assign dbg_halt = 1'b0; // TODO: implement in Stage 2

endmodule

```

Important: don't worry about halt detection yet. In Stage 2 you'll add proper branch logic and can detect jump-to-self correctly. For now, simulations will run until the cycle timeout — that's fine (but consider setting it to something short like 50 cycles depending on your program length).

Test with Hand-Encoded Hex

Before touching the encoder or compiler, verify your hardware works by hand-encoding a trivial program.

From your ISA spec, compute the bit pattern for these four instructions and convert to hex:


```
load the value 7 into register A
load the value 3 into register B
ADD register A and register B, store result in register C
HALT
```

Write the resulting hex values (one per line) into `tests/test_add.hex` and run:

```
python3 scripts/simulate.py my_cpu.simulate.yml
```

Expected register dump: - Register A = 0x00000007 - Register B = 0x00000003 - Register C = 0x0000000a (10)

If the result is correct, your fetch → decode → execute → writeback path is working end to end.

Now add one more instruction before the HALT and verify SUB works too:

```
load the value 7 into register A
load the value 3 into register B
ADD register A and register B, store in register C    ; C = 10
SUB register A and register B, store in register D    ; D = 4
HALT
```

Register D should contain 0x00000004. If ADD passes but SUB gives the wrong answer, the issue is in your ALU case statement, not the data path.

Encoder: Fill In Your Bit Layout

Open `tools/isa_config.py`. This is the ISA-specific configuration file that the assembler (`tools/asm.py`) imports. It contains your opcode tables, encoding functions, and any constants the assembler needs. The assembler script itself is generic and doesn't need editing.

A worked example first: ADD and MOV

Before you fill in your own values, here is a **complete** example so you can see exactly what goes into `isa_config.py` and how one line of assembly becomes a 32-bit word. Your ISA will differ — copy the *pattern*, not the numbers.

Example ALU instruction format (32-bit):

Bits	Field	Meaning
[31:24]	opcode	which operation (8 bits)
[23:20]	rd	destination register (r0-r15)
[19]	s1_imm	1 = src1 is a literal, 0 = a register
[18:15]	src1	source-1 register number (or 4-bit literal)
[14]	s2_imm	1 = src2 is a literal
[13:10]	src2	source-2 register number (or 4-bit literal)
[9:0]	—	unused

Opcode table — filled in for this example ISA:

```
# tools/isa_config.py
ALU_OPS = {
    'ADD': 0x01,
```

```
'SUB': 0x02,
'MOV': 0x03,
}
LOAD_IMM_OPS = { 'LI': 0x10 }
```

Encoding function – one masked, shifted |= per field:

```
def encode_alu(opcode_val, rd, src1_is_imm, src1, src2_is_imm, src2):
    """Pack a 32-bit ALU instruction for the format in the table above."""
    word = (opcode_val & 0xFF) << 24 # [31:24] opcode
    word |= (rd & 0x0F) << 20 # [23:20] destination register
    word |= (src1_is_imm & 0x01) << 19 # [19] src1-is-immediate flag
    word |= (src1 & 0x0F) << 15 # [18:15] src1 register / literal
    word |= (src2_is_imm & 0x01) << 14 # [14] src2-is-immediate flag
    word |= (src2 & 0x0F) << 10 # [13:10] src2 register / literal
    return word
```

Now encode three real instructions – mask + shift does all the work:

Assembly	encode_alu(...) call	Result
ADD r3, r1, r2	encode_alu(0x01, 3, 0, 1, 0, 2)	0x01308800
ADD r3, r1, #5	encode_alu(0x01, 3, 0, 1, 1, 5)	0x0130D400
MOV r4, r1	encode_alu(0x03, 4, 0, 1, 0, 0)	0x03408000

MOV needs only one source, so it reads src1 and ignores src2 (pass 0). Reading ADD r3, r1, r2 = 0x01308800 back out, field by field:

Field	Binary	Value
opcode	00000001	0x01 → ADD
rd	0011	r3
s1_imm	0	register
src1	0001	r1
s2_imm	0	register
src2	0010	r2
unused	0000000000	—

That's the whole job: choose a layout, list your opcodes, and write one &-mask + <<-shift per field. Now do the same for **your** ISA below.

1. Update the Opcode Tables

Replace the placeholder values with your actual opcode numbers from `opcodes.vh`:

```
# tools/isa_config.py – edit these to match YOUR ISA
```

```
ALU_OPS = {
    'ADD': ??, # your OP_ADD value
```

```

    'SUB': ??,    # your OP_SUB value
    'MOV': ??,    # your OP_MOV value
}

LOAD_IMM_OPS = {
    'LI':  ??,    # your OP_LI value
}

BRANCH_OPS = {
    'JMP': ??,    # needed for HALT – add in Stage 2 with real branches
}

```

2. Update the Encoding Functions

The config file contains functions that pack fields into a 32-bit word. **Edit the bit positions to match your encoding:**

```

def encode_alu(opcode_val, rd, src1_is_imm, src1, src2_is_imm, src2):
    """
    Pack an ALU instruction. Edit the bit shifts to match YOUR spec.

    Example – if your ALU format is:
        [31:28]=opcode [27:24]=rd [23:20]=rs1 [19:16]=rs2 ...
    then:
        word = (opcode_val & 0xF) << 28
        word |= (rd          & 0xF) << 24
        ...
    """
    word = 0
    # YOUR BIT LAYOUT HERE – one line per field
    return word

def encode_load_imm(opcode_val, rd, imm):
    """Pack a load-immediate instruction. Edit to match YOUR spec."""
    word = 0
    # YOUR BIT LAYOUT HERE
    return word

```

3. Validate

Encode the same program you hand-computed earlier and compare the output:

```

python3 tools/asm.py tests/test_add.asm -o build/test_add_enc.hex
diff build/test_add_enc.hex tests/test_add.hex

```

The diff should be empty. If any line differs, that instruction's encoding is wrong. Add a temporary print to your encoder showing each field value as it packs them, and compare field-by-field against your ISA spec.

Once the encoder output matches your hand-encoding, run the simulation again using the encoder's output instead of the hand-crafted file – the results must be identical:

```
python3 scripts/simulate.py my_cpu.simulate.yml
```

Register C = 0x0000000a, register D = 0x00000004.

Compiler Backend: First Build

You're teaching LLVM about your ISA. For Stage 1, the backend needs to: - Know what registers exist and which are allocatable - Know how to emit ADD, SUB, MOV instructions - Know how to load a constant into a register (LI) - Know the calling convention (which registers hold arguments and the return value) - Have a stub prologue/epilogue (no stack yet — that's Stage 3)

Files to Edit

MYISARegisterInfo.td Define your registers. Use your register names and hardware encodings from your ISA spec. Replace N in bits<N> and {N-1:0} with the number of bits required to represent a register in your design (so 4 for 16-registers, 5 for 32 and so on). Use your ISA name in this naming too.

```
// Adjust encoding width to match your register address bits
class MYISAReg<bits<N> Enc, string n> : Register<n> {
  let HWEncoding{N-1:0} = Enc;
  let Namespace = "MYISA";
}

// Declare each special purpose register.
def R0 : MYISAReg<0, "r0">;
def R1 : MYISAReg<1, "r1">;
// ... all non-allocatable registers ...

// The allocatable register class.
// List caller-saved registers first (the allocator prefers them),
// then callee-saved. Omit any reserved special-purpose registers
// (zero, PC, SP, LR — the compiler must never touch these freely).
def GPR : RegisterClass<"MYISA", [i32], 32,
  (add /* your caller-saved regs */,
    /* your callee-saved regs */);
```

MYISAIInstrFormats.td Describe your instruction bit layouts as TableGen classes. Each format becomes a class that your instruction definitions inherit from. The bit field names and positions must match your ISA spec exactly — this is what the assembly printer uses to output the right encoding.

Example: Your **InstrFormats.td* (e.g. *MYISAIInstrFormats.td*) defines format base classes such as *MYISAIInstType1* and *MYISAIInstType2*, each with a field *bits<32> Inst* that maps operand names to specific bit positions.

MYISAIInstrInfo.td Define each instruction. The key part is the **SelectionDAG pattern** — a declarative rule that says “when LLVM’s IR contains this operation, emit this instruction”:

```
// Pattern syntax: [(set destination, (ir_operation source1, source2))]
// The ir_operation names come from LLVM's SelectionDAG node vocabulary.

// ADD: matches the 'add' IR node
def ADD : YourALUFormat<YOUR_ADD_OPCODE,
  (outs GPR:$rd), (ins GPR:$rs1, GPR:$rs2),
  "ADD\t$rd, $rs1, $rs2",
  [(set GPR:$rd, (add GPR:$rs1, GPR:$rs2))]>;

// SUB: matches 'sub'
```

```

def SUB : YourALUFormat<YOUR_SUB_OPCODE,
  (outs GPR:$rd), (ins GPR:$rs1, GPR:$rs2),
  "SUB\t$rd, $rs1, $rs2",
  [(set GPR:$rd, (sub GPR:$rs1, GPR:$rs2))]>;

// MOV: a register copy – matches moving a value between registers
def MOV : YourALUFormat<YOUR_MOV_OPCODE,
  (outs GPR:$rd), (ins GPR:$rs1),
  "MOV\t$rd, $rs1",
  []>; // Matched by copyPhysReg in InstrInfo.cpp, not a TableGen pattern

// LI: load a small immediate constant.
// 'uimm5' here is a custom ImmLeaf – you need to define it in InstrInfo.td
// or a shared .td file. It restricts the pattern to constants that fit in 5 bits.
// Example definition:
//   def uimm5 : ImmLeaf<i32, [{ return Imm >= 0 && Imm < 32; }]>;
// Adjust the range to match your immediate field width.
def LI : YourImmFormat<YOUR_LI_OPCODE,
  (outs GPR:$rd), (ins i32imm:$imm),
  "LI\t$rd, # $imm",
  [(set GPR:$rd, uimm5:$imm)]>; // ← replace uimm5 with your immediate leaf

```

Replace YourALUFormat, YOUR_ADD_OPCODE, etc. with your actual format class names and opcode values.

MYISACallingConv.td Specify which registers hold function arguments, return values, and which callee-saved registers must be preserved across calls. Use the register roles you decided on in Stage 0:

```

def CC_MYISA : CallingConv<[
  // Arguments passed in registers (in order)
  CCIfType<[i32], CCAssignToReg[/ * your argument registers */>>,
  // Any further arguments spill to the stack
  CCIfType<[i32], CCAssignToStack<4, 4>>
]>;

def RetCC_MYISA : CallingConv<[
  // Return value register
  CCIfType<[i32], CCAssignToReg[/ * your return register */>>
]>;

// Registers the callee must preserve
def CSR_MYISA : CalleeSavedRegs<(add / * your callee-saved registers */>>;

```

MYISATargetMachine.cpp Set the data layout string. This tells LLVM your endianness, pointer width, and alignment requirements. For a 32-bit little-endian machine with 16-bit addresses:

```

// "e"      = little-endian
// "p:W:A"  = pointer is W bits wide, aligned to A bits
// "i32:32" = 32-bit integers are 32-bit aligned
// "n32"    = 32-bit is the native integer width
// "S32"    = stack is 32-bit aligned

```

```
static const char *DataLayout = "e-p:?:?-i32:32-n32-S32";
//                               ^^ pointer width = your address space width
```

MYISAISeLowering.cpp Tell LLVM which operations it should expand (synthesise from other operations) because your ISA doesn't have a direct instruction for them. Start permissive – expand everything you haven't implemented. You can tighten this later:

```
// In the constructor – expand operations you don't have instructions for
setOperationAction(ISD::BR_CC,    MVT::i32, Expand); // No branches yet (Stage 2)
setOperationAction(ISD::SELECT_CC, MVT::i32, Expand);
setOperationAction(ISD::MUL,      MVT::i32, Expand); // Unless you have MUL
setOperationAction(ISD::SDIV,     MVT::i32, Expand);
setOperationAction(ISD::UDIV,     MVT::i32, Expand);
setOperationAction(ISD::SREM,     MVT::i32, Expand);
setOperationAction(ISD::UREM,     MVT::i32, Expand);
// Add more Expand entries for anything else not yet implemented
```

MYISAFrameworkLowering.cpp Empty for now – no stack operations yet. Stage 3 fills this in:

```
void MYISAFrameworkLowering::emitPrologue(MachineFunction &MF,
                                           MachineBasicBlock &MBB) const {
    // Stage 3: save LR, save callee-saved regs, allocate frame
}
void MYISAFrameworkLowering::emitEpilogue(MachineFunction &MF,
                                           MachineBasicBlock &MBB) const {
    // Stage 3: deallocate frame, restore regs, RET
}
```

MYISAIInstrInfo.cpp Implement `copyPhysReg` using your `MOV` instruction. This is what the register allocator calls when it needs to move a value between physical registers:

```
void MYISAIInstrInfo::copyPhysReg(MachineBasicBlock &MBB,
                                   MachineBasicBlock::iterator I,
                                   const DebugLoc &DL,
                                   MCRegister DestReg, MCRegister SrcReg,
                                   bool KillSrc) const {
    BuildMI(MBB, I, DL, get(MYISA::MOV), DestReg)
        .addReg(SrcReg, getKillRegState(KillSrc));
}
```

Build

```
python3 scripts/build_compiler.py my_cpu.build-compiler.yml
```

Takes ~5–10 minutes. If anything fails, add `-v` for verbose output to see the exact CMake/ninja command that errored.

Verify your target was registered:

```
/opt/llvm/bin/llc --version # should list "myisa" in registered targets
```

Before you start editing backend files, read the `INSTALL.md` in your backend directory. It has a glos-

sary of every LLVM concept you'll encounter, a file-by-file reference, and a walkthrough of which files to edit and in what order. It exists specifically to save you time on the backend work.

Test: Compile and Run

The Test Program

```
// tests/test_add.c
__attribute__((noinline))
int add(int a, int b) {
    return a + b;
}
```

Startup Arguments

The compile script prepends a small `_start` stub that initialises the stack pointer and calls your entry function. The `args` field in `compile.yml` controls what values are loaded into your argument registers before the call:

```
startup:
  entry: main
  args: [21, 21]  # loaded into your first two argument registers
```

So with `args: [21, 21]`, your first argument register gets 21 and your second gets 21. The function should return 42 into your return register.

Compile and Simulate

```
python3 scripts/compile.py my_cpu.compile.yml
```

Update `my_cpu.simulate.yml` to verify the result ends up in your return register:

```
verify:
  - register: r??  # your return register
    value: 42
```

```
python3 scripts/simulate.py my_cpu.simulate.yml
```

If you see `Result: ALL 1 checks PASSED` — C code is running on hardware you designed.

Debugging

Problem	Where to look
Simulation produces wrong register values	Open the VCD in GTKWave. Trace <code>pc</code> , <code>instr</code> , decoder outputs, and <code>wr_data</code> cycle by cycle until you find where the value goes wrong.
Encoder output doesn't match hand-encoding	Check your bit shifts one field at a time. Add a <code>print(f"opcode at bits ??: {(word >> ??) & mask:#x}")</code> to verify each field independently.
<code>llc --version</code> doesn't show your target	The target wasn't registered in LLVM — check <code>LLVM_ALL_TARGETS</code> in <code>llvm/CMakeLists.txt</code> .

Problem	Where to look
<code>llc</code> crashes with LLVM ERROR: Cannot select	A pattern is missing – the compiler couldn't find an instruction for some IR operation. Check that ADD, SUB, and LI have valid patterns in <code>InstrInfo.td</code> .
Register dump shows all zeros after halt	<code>reg_write_en</code> is never high – check your decoder's control signal logic.
Wrong value in return register	Check the calling convention in <code>CallingConv.td</code> matches the register you're checking in <code>simulate.yml</code> .

Milestone Checklist

- ☐ Hand-encoded ADD test passes (correct value in result register)
- ☐ Encoder output matches hand-encoding for the same instruction
- ☐ LLVM backend builds; `llc --version` lists your target
- ☐ `add()` function compiles to valid assembly (inspect the `.s` file)
- ☐ Full pipeline: `compile.py` → `simulate.py` reports PASS

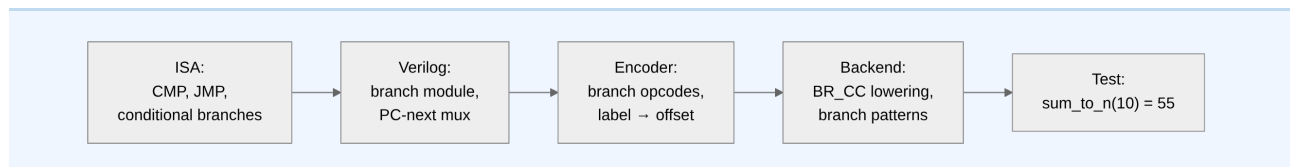
What's Next

- **Stage 2: Control Flow** – add CMP and branches, compile `sum_to_n(10) = 55`

Stage 2: Control Flow

Goal: Compile `int sum_to_n(int n) { int t=0; for(int i=1;i<=n;i++) t+=i; return t; }` and run it.

Your CPU can do arithmetic. Now it needs to make decisions and loop. This stage adds comparison and conditional branches – everything the compiler needs to generate `if/else`, `for`, and `while` constructs.



ISA: New Instructions

Add these to your ISA spec and opcodes.vh:

Instruction	What it does
CMP (or similar)	Compares two values; stores a result your branches can test
JMP target	Unconditional jump
Branch-if-equal	Jump if the last comparison was equal
Branch-if-not-equal	Jump if not equal
Branch-if-less	Jump if less than (signed)
Branch-if-greater	Jump if greater than (signed)

You don't need all four conditional branches immediately – `JMP`, `branch-if-equal`, and `branch-if-not-equal` are enough to compile most loops. Add the signed comparisons when you see the compiler needing them.

Two Decisions to Make

1. How do you encode branch targets?

A branch needs to specify where to jump to. Two approaches: – **PC-relative offset** – the address field holds a signed offset from the current PC. Compact, relocatable, and what the compiler naturally produces. The hardware computes `pc + offset` for the target. > **Important:** be precise about what "current PC" means. If offset is relative to the branch instruction's own address, then `offset = 0` jumps to itself (HALT). If offset is relative to the *next* instruction's address (PC+1), then `offset = -1` jumps to itself. **Pick one and write it down.** Assemblers and hardware must agree – a one-off error here is the most common cause of loops that run once and then fall through, or jump one instruction too far. – **Absolute address** – the address field holds a literal address. Simpler to implement (no arithmetic on the target), but the compiler needs to know absolute addresses at compile time.

Many ISAs support both, using a flag bit to select. Decide what makes sense for your encoding and document it.

2. How does the CPU know whether to take a branch?

The branch condition must come from somewhere. Common approaches: – **Dedicated condition register** – a specific register holds the result of the last `CMP` instruction; branches test it. Simple to implement: `CMP` just stores `rs1 - rs2` into that register, and branches check whether it's zero, negative, or

positive. - **Flag bits** – a status register with individual Z/N/C/V bits set automatically by ALU operations. More traditional (ARM/x86-style) but adds a separate register and extra decoder complexity. - **Compare-and-branch** – the branch instruction itself takes two register operands, compares them, and jumps. No separate CMP needed. This is how RISC-V works.

Pick one, document it in your ISA spec, and stick with it.

Update opcodes .vh

```
// Add your comparison and branch opcodes
`define OP_CMP    ??    // comparison operation (Type 1/ALU or its own type)
`define OP_JMP    ??
`define OP_BEQ    ??    // (or JZ, or whatever you've named it)
`define OP_BNE    ??
`define OP_BLT    ??
`define OP_BGT    ??
```

Verilog: Branch Logic

The key change is replacing the unconditional `pc_next = pc + 1` with logic that can redirect the PC.

Branch Condition

First, decide how your condition register is updated. If you chose the dedicated condition register approach, CMP is just subtraction written to a specific register – you may already have this from your ALU if you assign the result register correctly in assembly. The condition is then read from that register each cycle.

If you chose flags, you'll need a small flags register updated by the ALU.

Branch Module

Create a module that computes `pc_next` given the current PC, the condition register value, and the decoded instruction:

```
// rtl/branch.v
`include "config.vh"
`include "opcodes.vh"

module branch (
    input wire [`ADDR_MSB:0] pc_current,
    input wire [`DATA_MSB:0] condition,    // condition register / flags
    // Decoded fields from the instruction
    input wire [?:0] branch_opcode,
    input wire [?:0] branch_target_field,
    input wire is_branch_instr,
    input wire is_alu_instr,

    output wire [`ADDR_MSB:0] pc_next,
    output wire branch_taken
);
    // Step 1: Determine if condition is met
    wire cond_zero = (condition == 0);
    wire cond_neg  = condition[`DATA_MSB];    // sign bit for signed comparison
    wire cond_pos  = !cond_neg && !cond_zero;
```

```

wire take_branch;
// Assign take_branch based on the opcode and condition flags.
// For each conditional branch opcode, check the relevant condition bit.

assign branch_taken = is_branch_instr && take_branch;

// Step 2: Compute the target address.
// Be consistent with your assembler: if your encoder sets
//   offset = target_label_address - branch_instruction_address
// then the hardware must compute: target = pc_current + offset.
// If your encoder sets
//   offset = target_label_address - (branch_instruction_address + 1)
// then the hardware must compute: target = (pc_current + 1) + offset.
// Choose one, document it in your ISA spec, and use it in both places.
wire [`ADDR_MSB:0] target = /* compute from branch_target_field and pc_current
*/;

// Step 3: PC-next mux
assign pc_next = branch_taken ? target : (pc_current + 1);

endmodule

```

Connect to the Top-Level CPU

In `cpu.v`, replace `assign pc_next = pc + 1` with the branch module's output, and wire in the condition register value.

You need to read the condition register value every cycle to pass to the branch module. There are two ways:

Option A – Hierarchical reference (quick hack): Reach directly into the register file array. Works, but bypasses the proper port interface:

```

wire [`DATA_MSB:0] condition_val = u_regfile.regs[YOUR_CONDITION_REG_NUMBER];

```

Option B – Third read port (cleaner): Add a third read port to your regfile module (`rd_addr_c` / `rd_data_c`). Wire it permanently to the condition register address. This is architecturally cleaner and avoids hierarchical references, at the cost of slightly more register file wiring.

Either works. Pick one and be consistent.

```

wire [`ADDR_MSB:0] pc_next_from_branch;
wire               branch_taken;

branch u_branch (
    .pc_current(pc),
    .condition(condition_val),
    // ... connect decoded fields from decoder ...
    .pc_next(pc_next_from_branch),
    .branch_taken(branch_taken)
);

assign pc_next = pc_next_from_branch;

```

Halt Detection

Now that branch logic is in place, halt is simply a branch taken to the current PC (jump-to-self):

```
assign dbg_halt = branch_taken && (pc_next == pc);
```

Encoder: Add Branch Support

Update Opcode Tables

Add your branch opcodes to the encoder:

```
BRANCH_OPS = {
    'JMP': ??,
    'BEQ': ??,    # or JZ, or whatever your mnemonic is
    'BNE': ??,
    'BLT': ??,
    'BGT': ??,
}
```

Branch Encoding

The encoder already does two-pass assembly, so label addresses are available in pass 2. When a branch references a label, compute the offset:

```
# In your encode_branch() function:
if target_name in labels:
    # PC-relative: compute offset between target and current instruction.
    # MUST match your hardware's calculation exactly.
    #
    # If hardware computes: target = pc_current + offset
    # then: offset = labels[target_name] - current_pc
    #
    # If hardware computes: target = (pc_current + 1) + offset
    # then: offset = labels[target_name] - (current_pc + 1)
    #
    # Use whichever matches the choice you documented in your ISA spec.
    offset = labels[target_name] - current_pc # adjust for your convention
    # encode offset as a signed two's-complement value in your address field
    mask = (1 << ADDR_BITS) - 1
    encoded_offset = offset & mask # handles negative offsets
correctly
else:
    # Absolute address or literal offset
    addr_val = parse_imm(target_name)
    # encode with your absolute addressing mode
```

Validate with a Hand-Written Loop

Write a simple assembly loop and run it through the encoder and simulator. Fill in the mnemonics for your ISA:

```
; Count from 0 to 5
    LI    rCounter, #0      ; counter = 0
    LI    rLimit,  #5      ; limit = 5
loop:  ADD    rCounter, rCounter, #1 ; counter++
```

```

CMP  rCond, rCounter, rLimit ; cond = counter - limit
BNE  loop           ; if not zero, loop
HALT

```

Expected result: rCounter = 0x00000005, rLimit = 0x00000005.

If the loop runs forever (simulation times out), your branch offset is wrong – enable trace: true and find the cycle where the branch fires but goes to the wrong address. If the loop exits after one iteration, the comparison result is being treated as the wrong sign – check that BNE tests for non-zero, not non-negative.

Now add a second test to verify signed less-than branching works. This catches sign-extension bugs that the equality test above won't find:

```

; Subtract 3 repeatedly until the result goes negative
LI    rAcc, #10           ; acc = 10
loop: SUB  rAcc, rAcc, #3   ; acc -= 3
      CMP  rCond, rAcc, r0 ; cond = acc - 0
      BGT  loop           ; if acc > 0, loop again
      HALT

```

Starting from 10, subtracting 3 each time: 10 → 7 → 4 → 1 → -2. The loop should exit with rAcc = 0xffffffff (−2 in two's complement). If your CPU exits with rAcc = 0x00000001 instead, your BGT condition is testing unsigned rather than signed comparison.

Compiler Backend: Add Branch Support

MYISAIInstrInfo.td – New Instructions

Add CMP and the branch instructions. Branches don't usually have automatic TableGen patterns because they involve condition codes – mark them with empty pattern lists and handle them in C++: > **Note:** brtarget and bb(basic block) are operand types already defined by LLVM's Target.td (included via your root.td file). You don't need to define them yourself. If you get a TableGen error saying brtarget is undefined, check that your root MYISA.td includes the right LLVM includes.

```

// CMP – comparison writes to your condition register
def CMP : YourALUFormat<YOUR_CMP_OPCODE,
  (outs YOUR_CONDITION_REG_CLASS:$rd), (ins GPR:$rs1, GPR:$rs2),
  "CMP\t$rd, $rs1, $rs2",
  []>; // No automatic pattern – custom lowered

// Unconditional jump
def JMP : YourBranchFormat<YOUR_JMP_OPCODE,
  (outs), (ins brtarget:$target),
  "JMP\t$target",
  [(br bb:$target)]>;

// Conditional branches – no automatic patterns
def BEQ : YourBranchFormat<YOUR_BEQ_OPCODE,
  (outs), (ins brtarget:$target), "BEQ\t$target", []>;
def BNE : YourBranchFormat<YOUR_BNE_OPCODE,
  (outs), (ins brtarget:$target), "BNE\t$target", []>;
def BLT : YourBranchFormat<YOUR_BLT_OPCODE,

```

```
(outs), (ins brtarget:$target), "BLT\t$t$target", []>;
def BGT : YourBranchFormat<YOUR_BGT_OPCODE,
    (outs), (ins brtarget:$target), "BGT\t$t$target", []>;
```

MYISAISeLowering.cpp – Custom Branch Lowering

Change BR_CC from Expand to Custom:

```
setOperationAction(ISD::BR_CC, MVT::i32, Custom);
// Note: do NOT also set BRCOND to Custom unless your lowering uses it.
// BR_CC is the higher-level node (compare + branch as one unit).
// Setting both leads to double-lowering; Custom BR_CC is usually sufficient.
```

Then implement LowerBR_CC(). It receives one IR node saying “compare LHS and RHS with condition CC, jump to Dest if true.” Lower it into a CMP followed by the appropriate conditional branch:

```
SDValue MYISATargetLowering::LowerBR_CC(SDValue Op, SelectionDAG &DAG) const {
    // Op operands: chain, condition code, LHS, RHS, destination block
    ISD::CondCode CC = cast<CondCodeSDNode>(Op.getOperand(1))->get();
    SDValue LHS = Op.getOperand(2);
    SDValue RHS = Op.getOperand(3);
    SDValue Dest = Op.getOperand(4);
    SDLoc DL(Op);

    // 1. Emit CMP to compute LHS - RHS into the condition register
    // 2. Select the right branch opcode based on CC (see table below)
    // 3. Return the branch node chained to the CMP
}
```

The condition code mapping from LLVM ISD::CondCode values to branch opcodes:

ISD::CondCode	Meaning	Your branch opcode
ISD::SETEQ	equal	BEQ / JZ
ISD::SETNE	not equal	BNE / JNZ
ISD::SETLT	signed less than	BLT / JLT
ISD::SETLE	signed less-or-equal	BLT or BEQ combined
ISD::SETGT	signed greater than	BGT / JGT
ISD::SETGE	signed greater-or-equal	BGT or BEQ combined

If your ISA doesn't have a direct instruction for SETLE / SETGE, you have two options: synthesise them from two branches, or ask LLVM to expand them into SETLT/SETEQ combinations (add setCondCodeAction(ISD::SETEQ, MVT::i32, Expand)).

Tip: LowerBR_CC in your MYISAISeLowering.cpp is where this happens. The condition code mapping (ISD::SETEQ → MYISAISD::BEQ, etc.) is the main thing to adapt.

Rebuild and Test

```
python3 scripts/build_compiler.py my_cpu.build-compiler.yml
```

Test with a simple conditional first – inspect the assembly output before running simulation:

```
/opt/llvm/bin/clang -emit-llvm -S -O1 -o /tmp/test.ll tests/test_abs.c
/opt/llvm/bin/llc -march=myisa -O0 -o /tmp/test.s /tmp/test.ll
cat /tmp/test.s
```

Does it emit CMP followed by a conditional branch? Are the branch directions correct?

Test: Compile and Run

Test 1 – Simple Conditional

```
// tests/test_abs.c
__attribute__((noinline))
int abs_val(int x) {
    if (x < 0) return -x;
    return x;
}
```

Test with a negative input (should return the positive value) and a positive input (should return unchanged).

Test 2 – Loop

```
// tests/test_sum.c
__attribute__((noinline))
int sum_to_n(int n) {
    int total = 0;
    for (int i = 1; i <= n; i++)
        total += i;
    return total;
}
```

Pass `n = 10` via the startup args. Expected result: 55.

Update `my_cpu.simulate.yml`:

```
verify:
  - register: r??    # your return register
    value: 55
```

```
python3 scripts/compile.py my_cpu.compile.yml
python3 scripts/simulate.py my_cpu.simulate.yml
```

Debugging

Problem	Where to look
Branch never taken	Is the condition register being written? Trace <code>wr_addr</code> and <code>wr_data</code> in GTKWave – verify CMP writes to the right register.
Branch always taken	Is <code>is_branch_instr</code> incorrectly asserted for non-branch instructions? Check your decoder's classification logic.

Problem	Where to look
Wrong branch target	Print <code>pc</code> , the raw offset field, and <code>pc_next</code> during simulation. Check sign-extension of the offset – a missing sign-extension is the most common cause.
Infinite loop that never halts	The branch offset is off by one – check whether your offset is “from current PC” or “from next PC”.
Compiler generates wrong branch direction	The condition code mapping in <code>LowerBR_CC</code> is inverted. Check <code>ISD : : SETLT</code> vs <code>ISD : : SETLE</code> etc.

GTKWave Tips for Branch Debugging

Add these signals and step through the loop iteration: - `pc`, `pc_next` - `branch_taken` - your condition register’s value - your decoded branch opcode

Find the clock cycle where the branch should fire and verify `branch_taken` goes high at the right time.

Milestone Checklist

- ☐ JMP works (can write a program that loops in hand-encoded hex)
- ☐ CMP writes to the condition register correctly
- ☐ At least branch-if-equal and branch-if-not-equal work
- ☐ Hand-written loop test produces correct result
- ☐ Encoder resolves labels to correct offsets (forward and backward)
- ☐ Compiler emits CMP + conditional branch for `if/else`
- ☐ `sum_to_n(10)` compiles and simulates to 55

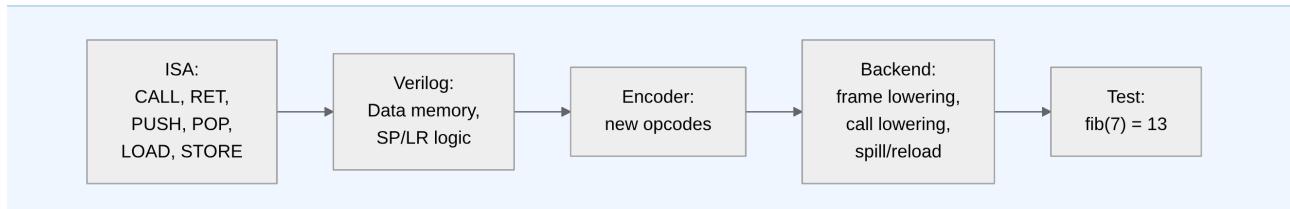
What’s Next

- **Stage 3: Function Calls** – add `CALL/RET`, stack operations, data memory → compile recursive `fib(7)`
= 13

Stage 3: Function Calls

Goal: Compile recursive `fib(7) = 13` in C and run it on your CPU. This is the victory lap.

Recursive Fibonacci exercises everything: function calls, stack frames, register saving, nested recursion, branching, and arithmetic. If this works end-to-end, your full stack is complete.



ISA: New Instructions

Instruction	What it does	Why you need it
CALL target	Saves return address, jumps to target	Function call
RET	Restores PC from saved return address	Return from function
PUSH rs	Saves a register to the stack	Preserving registers across calls
POP rd	Restores a register from the stack	Restoring registers on return
LOAD rd, addr	rd = mem[addr]	Reading from data memory
STORE rs, addr	mem[addr] = rs	Writing to data memory

Why All Six Together?

A leaf function (one that calls nothing else) only needs `CALL` + `RET`. But the moment functions call other functions, the link register gets overwritten at each call. To survive nested calls, you must `PUSH` the link register onto the stack before calling and `POP` it back before returning. `LOAD` and `STORE` (with `SP`-relative addressing) are also needed when the compiler spills registers to the stack.

Three Design Decisions

1. Where does `CALL` save the return address?

Options: - **Link register** – `CALL` writes `PC + 1` into a dedicated register (your `LR`). Fast, but that register must be saved to the stack before making any further calls. - **Push to stack** – `CALL` automatically pushes the return address onto the stack. Simpler calling convention, but the hardware is doing more.

Most RISC ISAs use a link register.

2. How do `PUSH` and `POP` modify the stack pointer?

You need to decide: does the stack grow up (`SP` increases) or down (`SP` decreases)? Convention is downward, but either works. Also decide whether `PUSH` pre-decrements `SP` before writing, or post-decrements after. These must be consistent between hardware, encoder, compiler, and the startup stub.

3. How do `LOAD/STORE` address memory?

For the compiler to spill registers to the stack, it needs `SP`-relative addressing: `LOAD rd, [SP + offset]`. You can implement this as a separate addressing mode, or as a fixed-format instruction where the address field is added to `SP` automatically.

Document all three decisions in your ISA spec before writing any Verilog.

Update opcodes.vh

```
`define OP_CALL    ??
`define OP_RET     ??
`define OP_PUSH    ??
`define OP_POP     ??
`define OP_LOAD    ??
`define OP_STORE   ??
```

Verilog: Data Memory & Stack**Data Memory Module**

Add a read/write data memory alongside your existing instruction memory:

```
// rtl/dmem.v
`include "config.vh"

module dmem #(
    parameter DEPTH = `DMEM_DEPTH
)(
    input wire          clk,
    input wire          wr_en,
    input wire [`ADDR_MSB:0] addr,
    input wire [`DATA_MSB:0] wr_data,
    output wire [`DATA_MSB:0] rd_data
);
    reg [`DATA_MSB:0] mem [0:DEPTH-1];

    always @(posedge clk)
        if (wr_en) mem[addr] <= wr_data;

    assign rd_data = mem[addr]; // asynchronous read
endmodule
```

Address alignment: If your data memory is byte-addressed (which the compiler expects) but word-wide (32-bit arrays), you cannot use the raw instruction address directly to index *mem*. A byte address like 0x0008 is the third 32-bit word: *mem*[0x0008 >> 2] = *mem*[2]. Strip the low two bits: *mem*[*addr*[ADDR_MSB:2]]. If your ISA only supports word-aligned accesses (no individual byte loads), this is fine. If you plan to support LOADB/STOREB later, you'll also need byte-lane selection.

Also add *WORD_BYTES* to your *config.vh* now — you'll need it in PUSH/POP logic:

```
`define WORD_BYTES (`DATA_WIDTH / 8) // bytes per word (e.g. 4 for 32-bit)
```

CALL / RET

CALL does two things in the same cycle: save the return address and change the PC.

In your branch module (or in a dedicated CALL/RET section of the CPU):

```
wire is_call = /* decode from opcode */;
wire is_ret  = /* decode from opcode */;

// On CALL: jump to target, and signal that LR should be written
```

```

assign branch_taken = is_jump || is_call || is_ret || /* conditional branches */;

assign branch_target =
    is_ret ? lr_value[`ADDR_MSB:0] : // return to saved address
    is_call ? /* target from instruction */ :
              /* same as before for other branches */;

// Signal to the top-level to write PC+1 into the link register
assign save_lr = is_call;

```

In the top-level CPU, add a write path for the link register:

```

wire [`ADDR_MSB:0] return_addr = pc + 1;

// Write return_addr into the link register on CALL
// (connect this to the register file write port, or use a dedicated LR register)

```

How you implement this depends on your register file interface. If LR is just a normal register (e.g., r3), you can use the normal write port with a mux. If you prefer a dedicated port, add one.

PUSH / POP

PUSH and POP are memory operations that also update the stack pointer. You need to: 1. Compute the effective memory address (SP before or after decrement, depending on your convention) 2. Either write to (PUSH) or read from (POP) data memory at that address 3. Update SP

Wire the SP register to a read port on the register file, or add a dedicated SP port if your design separates it. The SP update must happen on the same clock edge as the memory operation.

```

wire is_push = /* decode */;
wire is_pop  = /* decode */;

// Address used for the memory operation
wire [`ADDR_MSB:0] stack_addr =
    is_push ? (sp_value - WORD_BYTES) : // pre-decrement
    is_pop  ? sp_value : // current SP
              /* LOAD/STORE address from instruction */;

wire dmem_wr_en = is_push || is_store;

// SP update value
wire [`DATA_MSB:0] sp_new =
    is_push ? (sp_value - WORD_BYTES) :
    is_pop  ? (sp_value + WORD_BYTES) :
              sp_value; // unchanged

```

Replace WORD_BYTES with a define from your config.vh (e.g., DATA_WIDTH / 8).

Updated Write-Back Mux

LOAD and POP both write data memory contents to a register. Update the write-back mux:

```

wire is_load = /* decode */;

assign wb_data =
    is_alu_op ? alu_result :

```

```
is_li    ? /* sign/zero extended immediate */ :
(is_load || is_pop) ? dmem_rd_data :
           {`DATA_WIDTH{1'b0}};
```

```
assign reg_write_en = is_alu_op || is_li || is_load || is_pop;
```

Quick write-back mux test. Run this program before touching CALL/RET:

```
LI      rA, #0x55      ; rA = 0x55 (via LI path)
ADD     rB, rA, rA      ; rB = 0xAA (via ALU path)
STORE   rB, [0x0200]    ; mem[0x0200] = 0xAA
LI      rB, #0          ; clear rB
LOAD    rC, [0x0200]    ; rC = 0xAA (via memory path)
HALT
```

Expected: rA = 0x00000055, rB = 0x00000000, rC = 0x000000aa.

This exercises all three write-back sources in one program. If rC is zero but rB was 0xaa before the LI, your load write-back path is being masked by is_alu_op taking priority in the mux.

Test with Hand-Encoded Hex

Before touching the compiler, verify your hardware with these three hand-encoded programs in order. Each one isolates a specific mechanism.

Test 1 – LOAD and STORE.

Store a value to a known memory address and read it back:

```
LI      rA, #0xBEEF    ; rA = 0xBEEF
STORE   rA, [0x0100]    ; mem[0x0100] = 0xBEEF
LI      rA, #0          ; clear rA to prove the load is doing the work
LOAD    rB, [0x0100]    ; rB = mem[0x0100]
HALT
```

Expected: rA = 0x00000000, rB = 0x0000beef.

If rB is still zero, your data memory write enable or address decode is wrong. If rB contains 0x0000beef but with swapped bytes, you have an endianness bug in your byte-addressing logic.

Test 2 – simple CALL/RET (no stack needed):

```
LI      rArg, #6        ; argument = 6
CALL    double_it
HALT                      ; rResult should be 12
```

```
double_it:
ADD     rResult, rArg, rArg ; result = arg + arg
RET
```

Expected: your designated result register = 0x0000000c (12).

If execution never reaches HALT (timeout), RET is jumping to the wrong address – print the link register value before RET fires and verify it equals the address of the HALT instruction.

Test 3 – nested CALL (exercises PUSH/POP of LR):

```

    LI    rA, #0           ; rA = 0
    CALL  add_two          ; should add 2 to rA via two calls
    HALT                    ; rA should be 2

```

add_two:

```

    PUSH  rLR
    CALL  add_one          ; rA += 1
    CALL  add_one          ; rA += 1 again
    POP   rLR
    RET

```

add_one:

```

    ADD   rA, rA, #1
    RET

```

Expected: rA = 0x00000002.

If rA is correct but execution crashes or doesn't return from add_two, check the SP value in GTKWave — it should decrease by one word on each PUSH and increase by one word on each POP, ending at its initial value when HALT is reached. If SP drifts, your pre/post-decrement direction or word size is wrong.

Encoder: Add New Opcodes

Update tools/isa_config.py with the new opcodes and encoding cases:

```

# Add to your existing opcode tables
BRANCH_OPS.update({'CALL': ??})    # CALL is branch-like (has a target)

MISC_OPS = {
    'RET':  ??,
    'PUSH': ??,
    'POP':  ??,
    'LOAD': ??,
    'STORE': ??,
}

```

Add encoding for each in your encoding functions. For CALL, handle the same label/offset logic as other branches. For PUSH/POP, the relevant field is the register operand. For LOAD/STORE, the relevant fields are the register and the address.

Compiler Backend: Frame Lowering & Call Support

This is the most involved backend change, but also the most systematic. You're implementing three things:

1. **Call lowering** — how to set up arguments and receive return values when calling a function
2. **Frame lowering** — what instructions to emit at function entry (prologue) and exit (epilogue)
3. **Register spilling** — how to save/reload registers to/from the stack when the register allocator runs out

MYISAIInstrInfo.td – New Instructions

```

def CALL : YourBranchFormat<YOUR_CALL_OPCODE,
    (outs), (ins calltarget:$target), "CALL\t$t$target", []>;

def RET : YourControlFormat<YOUR_RET_OPCODE,
    (outs), (ins), "RET", []>;

def PUSH : YourStackFormat<YOUR_PUSH_OPCODE,
    (outs), (ins GPR:$rs), "PUSH\t$rs", []>;

def POP : YourStackFormat<YOUR_POP_OPCODE,
    (outs GPR:$rd), (ins), "POP\t$rd", []>;

def LOAD : YourMemFormat<YOUR_LOAD_OPCODE,
    (outs GPR:$rd), (ins mem_operand:$addr),
    "LOAD\t$rd, $addr",
    [(set GPR:$rd, (load addr:$addr))]>;

def STORE : YourMemFormat<YOUR_STORE_OPCODE,
    (outs), (ins GPR:$rs, mem_operand:$addr),
    "STORE\t$rs, $addr",
    [(store GPR:$rs, addr:$addr)]>;

```

MYISAFrameworkLowering.cpp – Prologue and Epilogue

Now implement the real function entry and exit sequences. The prologue runs at the start of every function; the epilogue runs before every return.



```

void MYISAFrameworkLowering::emitPrologue(MachineFunction &MF,
                                           MachineBasicBlock &MBB) const {
    MachineBasicBlock::iterator MBBI = MBB.begin();
    MachineFrameInfo &MFI = MF.getFrameInfo();
    const MYISAIInstrInfo &TII =
        *static_cast<const MYISAIInstrInfo *>(MF.getSubtarget().getInstrInfo());
    DebugLoc DL;

    // 1. Save the link register
    BuildMI(MBB, MBBI, DL, TII.get(MYISA::PUSH))
        .addReg(MYISA::YOUR_LR_REG);

    // 2. Save each callee-saved register this function actually uses.
    //    getCalleeSavedInfo() is a method on MachineFunction, not
    //    MachineFrameInfo.
    for (const auto &CSI : MF.getCalleeSavedInfo()) // ← note: MF, not MFI
        BuildMI(MBB, MBBI, DL, TII.get(MYISA::PUSH))
            .addReg(CSI.getReg());

    // 3. Allocate the stack frame.

```

```

// MFI.getStackSize() is set by the register allocator before emitPrologue
// is called – don't cache it before this point.
int FrameSize = MFI.getStackSize();
// If your immediate field is small (e.g. 5 bits = max 31), emit multiple
// SUBs. Most ISAs can handle typical frame sizes in one instruction.
if (FrameSize > 0) {
    // Simple version: one SUB. If FrameSize > your max immediate,
    // you'll see a TableGen/ISel error; add a loop here if needed.
    BuildMI(MBB, MBB, DL, TII.get(MYISA::SUB), MYISA::YOUR_SP_REG)
        .addReg(MYISA::YOUR_SP_REG).addImm(FrameSize);
}
}

```

Replace `MYISA::YOUR_LR_REG`, `MYISA::YOUR_SP_REG`, and `MAX_IMMEDIATE` with your actual register and immediate constants.

MYISAISeLowering.cpp – Call Lowering

This is the most involved part of Stage 3. You need to implement three methods:

- **LowerCall()** – runs when the IR contains a function call; copies arguments to physical registers, emits CALL, copies the return value out
- **LowerFormalArguments()** – runs at the start of every function; copies physical argument registers into virtual registers for the compiler to use freely
- **LowerReturn()** – runs at every return; copies the return value into the physical return register and emits RET

Tip: Your `MYISAISeLowering.cpp` is where all three live. The argument register list comes from the calling convention in `MYISACallingConv.td`. Adapt the register names and CALL/RET opcodes for your design. The overall structure – creating `InVals/OutVals` lists, walking the calling convention using `CCState`, building copy chains – does not change between ISAs.

Your backend's `INSTALL.md` (copied from the template) also has a detailed walkthrough of these three functions in its porting guide.

Before writing any C++, sketch the call sequence your ISA needs for a simple non-leaf function: PUSH LR, set up arguments, CALL, retrieve the return value, POP LR. Your `LowerCall()`, `LowerFormalArguments()`, and `LowerReturn()` must together produce that sequence.

MYISARegisterInfo.cpp – Frame Index Elimination

When the compiler allocates a stack slot (for a local variable or spilled register), it refers to it with an abstract “frame index.” Before code generation is complete, these must be converted to concrete `SP + offset` references.

```

void MYISARegisterInfo::eliminateFrameIndex(
    MachineBasicBlock::iterator II,
    int SPAdj, unsigned FIOperandNum,
    RegScavenger *RS) const {
    MachineInstr &MI = *II;
    MachineFunction &MF = *MI.getMF();
    int FrameIdx = MI.getOperand(FIOperandNum).getIndex();
    int Offset = MF.getFrameInfo().getObjectOffset(FrameIdx)
        + MF.getFrameInfo().getStackSize()
        + SPAdj;
    // Replace the frame index operand with SP + Offset
}

```

```

    MI.getOperand(FIOperandNum).ChangeToRegister(MYISA::YOUR_SP_REG, false);
    MI.getOperand(FIOperandNum + 1).ChangeToImmediate(Offset);
}

```

MYISAIInstrInfo.cpp – Spill and Reload

The register allocator calls these when it needs to move a register to/from the stack:

```

void MYISAIInstrInfo::storeRegToStackSlot(MachineBasicBlock &MBB,
                                           MachineBasicBlock::iterator I,
                                           Register SrcReg, bool IsKill,
                                           int FrameIdx,
                                           const TargetRegisterClass *RC,
                                           const TargetRegisterInfo *TRI,
                                           Register VReg) const {
    BuildMI(MBB, I, DebugLoc(), get(MYISA::STORE))
        .addReg(SrcReg, getKillRegState(IsKill))
        .addFrameIndex(FrameIdx).addImm(0);
}

void MYISAIInstrInfo::loadRegFromStackSlot(MachineBasicBlock &MBB,
                                           MachineBasicBlock::iterator I,
                                           Register DestReg,
                                           int FrameIdx,
                                           const TargetRegisterClass *RC,
                                           const TargetRegisterInfo *TRI,
                                           Register VReg) const {
    BuildMI(MBB, I, DebugLoc(), get(MYISA::LOAD), DestReg)
        .addFrameIndex(FrameIdx).addImm(0);
}

```

Rebuild

```
python3 scripts/build_compiler.py my_cpu.build-compiler.yml
```

Test: The Gold Standard

Don't jump straight to recursive Fibonacci. Build up through these five tests in order – each one exercises one more piece of the calling convention:

```

// Test 1: leaf function (no stack needed at all)
__attribute__((noinline))
int double_it(int x) { return x + x; }

// Test 2: non-leaf function (needs to save LR)
__attribute__((noinline))
int add_via_call(int a, int b) { return double_it(a) + b; }

// Test 3: function with locals (needs stack frame allocation)
__attribute__((noinline))
int with_local(int x) { int y = x + 1; return double_it(y); }

// Test 4: simple recursion (needs everything)

```



```
__attribute__((noinline))
int factorial(int n) { if (n <= 1) return 1; return n * factorial(n - 1); }

// Test 5: double recursion (the final boss)
__attribute__((noinline))
int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
int main(void) { return fib(7); }
```

For each test, inspect the generated assembly (.asm file in build/compile/) before running simulation. The assembly is your primary debugging tool here.

For each test, before simulating, verify in the .asm file: - Test 1(leaf): no PUSH/POP, no stack allocation, just args → result → RET - Test 2(non-leaf): PUSH LR at entry, POP LR before RET - Test 3(locals): stack allocated (SP decremented in prologue, incremented in epilogue), locals at SP+offset - Test 4 (simple recursion): same as Test 3, plus a CALL inside the function body - Test 5 (double recursion): two CALLs with the result of the first used in the second

If the assembly looks wrong for a test, fix the prologue/epilogue/lowering before running simulation. Simulating with incorrect assembly makes debugging much harder.

Update my_cpu.simulate.yml for the final test:

```
verify:
  - register: r??      # your return register
    value: 13          # fib(7) = 13
options:
  max-cycles: 200000   # recursive fib takes more cycles than a loop
```

```
python3 scripts/compile.py my_cpu.compile.yml
python3 scripts/simulate.py my_cpu.simulate.yml
```

Debugging

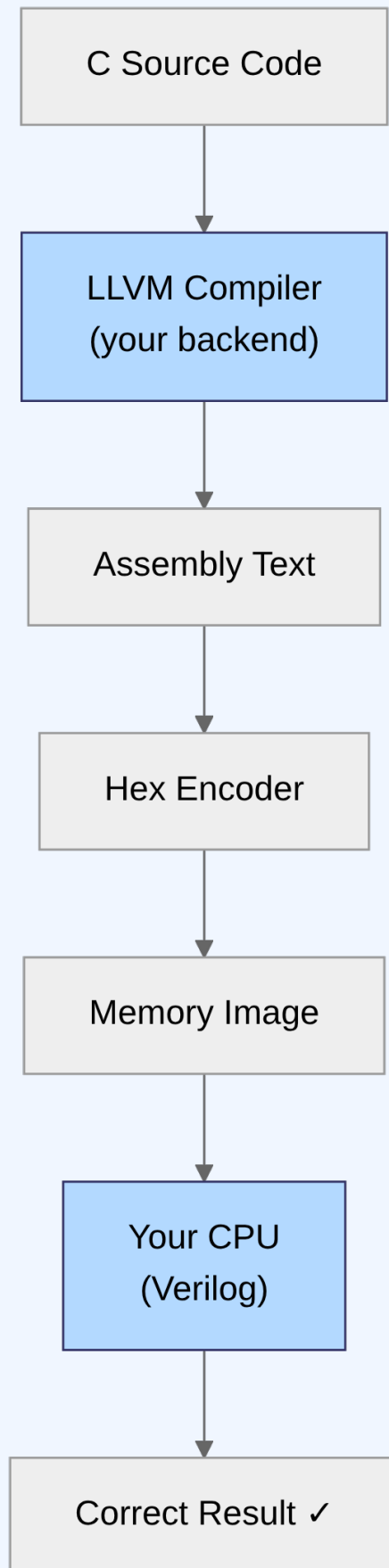
Problem	Where to look
Leaf function works, nested call corrupts	LR not being saved in prologue – inspect .asm output for PUSH LR at function entry
Stack corruption	SP increment/decrement direction or size is wrong – trace SP value in GTKWave through a PUSH/POP pair
Wrong return value for a non-recursive call	Check LowerReturn() in lseLowering – is the return value being put in the right register?
Spill/reload causes wrong value	Frame offset calculation in eliminateFrameIndex – add a debug print of the computed offset
fib(7) gives wrong answer	Reduce to fib(1), fib(2), fib(3) to find where the recursion breaks
Simulation times out on fib	max-cycles too low, or the CPU is looping – check that RET restores the correct PC

Milestone Checklist

- ☐ CALL/RET work for a leaf function
 - ☐ PUSH/POP work (verified with hand-encoded hex)
 - ☐ Nested CALL works (LR saved to stack and restored correctly)
 - ☐ LOAD/STORE work (data memory reads and writes)
 - ☐ Test 1(leaf function) compiles and runs correctly
 - ☐ Test 2 (non-leaf) compiles and runs correctly
 - ☐ Test 3(locals) compiles and runs correctly
 - ☐ Test 4 (simple recursion) compiles and runs correctly
 - ☐ **fib(7) = 13 – full pipeline PASS**
-

You're Done!

If recursive Fibonacci passes, you have a complete system:



See **Stretch Goals** for FPGA synthesis, performance tuning, additional instructions, and peripherals.

Stage 4: Stretch Goals

You have a working CPU that compiles and runs recursive C programs. Everything from here is bonus – pick what interests your team.

Additional ALU Operations

Add more operations to make compiled code faster (fewer instructions per operation):

Instruction	Impact	Effort
AND, OR, XOR	Bitwise logic (compiler currently expands these)	Low
NOT, NEG	Bitwise complement, two's complement negate	Low
SHL, SHR, SAR	Shifts (compiler synthesises from adds otherwise)	Low
MUL	Hardware multiply (saves ~30 cycles per multiply)	Medium
LUI	Load upper immediate (load 32-bit constants in 2 instructions)	Low

For each: add to `opcodes.vh`, add to ALU case statement, add to encoder, add TableGen pattern in `InstrInfo.td`. Rebuild backend.

FPGA Synthesis

Deploy your CPU to real hardware.

Using the Script

```
python3 scripts/synthesize.py my_cpu.synthesize.yml
```

```
# my_cpu.synthesize.yml
synthesize:
  output-dir: ./build/quartus
  device: 5CSEMA5F31C6      # Your FPGA part number
  top: cpu
  rtl:
    base-dir: ./rtl
    files:
      - my_pkg.v
      - cpu.v
  include-testbench: false
```

Open-Source Alternative (No Quartus)

```
yosys -p "read_verilog -I rtl/include rtl/*.v; synth -top cpu; stat"
```

This gives gate count statistics for comparing designs between teams.

Key Metrics

Metric	What it means
Logic Elements / ALMs	Area (how much FPGA you use)
Fmax	Maximum clock frequency
Memory Bits	Block RAM usage
Cycles $\times$ LUTs	Energy proxy (lower is better)

FPGA Top-Level Wrapper

Map your CPU's ports to board peripherals:

```
module cpu_fpga_top (
    input wire      CLOCK_50,
    input wire [3:0] KEY,
    output wire [9:0] LEDR,
    output wire [6:0] HEX0, HEX1, HEX2, HEX3
);
    wire rst_n = KEY[0];

    cpu #(.IMEM_INIT_FILE("program.hex")) u_cpu (
        .clk(CLOCK_50),
        .rst_n(rst_n),
        .dbg_pc(/* → HEX displays */),
        .dbg_halt(LEDR[0])
    );
endmodule
```

Performance Tuning

Reduce Cycle Count

- Wider immediate fields → fewer instructions to load constants
- Add MUL instruction → massive improvement for arithmetic-heavy code
- Implement more addressing modes → fewer instructions for memory access

Increase Fmax

- Identify the critical path (usually through the ALU or branch logic)
- Consider pipelining (see below)
- Use synchronous memory reads (adds latency but simplifies timing)

Reduce Area

- Fewer registers = smaller mux trees
- Remove unused instructions from the ALU
- Narrower data path if your programs don't need 32 bits

Pipelining

Split execution into stages for higher throughput:



2-stage pipeline (easiest): - Stage 1: Fetch + Decode - Stage 2: Execute + Write-back

3-stage pipeline: - Stage 1: Fetch - Stage 2: Decode + Register Read - Stage 3: Execute + Write-back

Hazards to handle: - **Data hazard:** instruction reads a register that the previous instruction is writing → add forwarding or insert a stall - **Control hazard:** branch destination unknown until execute stage → insert a bubble (or predict not-taken)

Peripherals

UART TX (Serial Output)

Add a memory-mapped UART transmitter so your CPU can print characters:

Address 0x8010: UART TX data (write a byte to send it)

Address 0x8014: UART TX status (bit 0 = busy)

A minimal UART TX at 115200 baud on a 50 MHz clock is ~50 lines of Verilog.

GPIO (LEDs / Switches)

Memory-mapped GPIO:

Address 0x8020: LED output register (write to set LEDs)

Address 0x8024: Switch input register (read to get switch state)

Timer / Interrupt

Add a countdown timer that fires an interrupt when it reaches zero. This requires: - An interrupt controller - A way to save/restore CPU state on interrupt entry/exit - An interrupt vector table

This is a significant extension but teaches real embedded systems concepts.

Byte-Addressable Memory

If your data memory is word-addressed, consider adding byte-level access (LOADB/STOREB). This is needed for string operations and char arrays in C. Requires alignment logic in the memory interface.

Presentation Tips

For the final demo:

1. **Show it working** – compile and run a program live (or show the simulation output)
2. **One-page architecture diagram** – block diagram of your CPU with data paths
3. **ISA encoding table** – one-page summary of your instruction formats
4. **Metrics** – cycle count for fib(7), gate count if synthesised, Fmax
5. **What you learned** – what design decisions worked? What would you change?

If you have FPGA hardware: a live demo with visible output (LEDs, 7-segment showing the result) is more impressive than any slide.

Reference

Resource	Where
LLVM backend template	resources/software/template/
LLVM backend porting guide	resources/software/template/llvm-backend/INS
Simulation script	scripts/simulate.py
Compile script	scripts/compile.py
Synthesis script	scripts/synthesize.py

Milestone Checklist

Track your progress through the core stages:

- ☐ **Stage 0:** ISA designed and documented; project scaffold created
- ☐ **Stage 1:** ADD/SUB/MOV work; first program runs in simulation
- ☐ **Stage 1:** Encoder + compiler backend build; C compiles to hex
- ☐ **Stage 2:** Conditional branches work; loops execute correctly
- ☐ **Stage 2:** Compiler generates branch code; loop test passes
- ☐ **Stage 3:** CALL/RET + PUSH/POP work; stack operations verified
- ☐ **Stage 3:** Recursive `fib(7) = 13` compiles and runs correctly

Stretch milestones:

- ☐ Additional ALU ops (AND, OR, SHL, MUL, etc.)
- ☐ Assembler written and tested independently
- ☐ FPGA synthesis completed; resource metrics recorded
- ☐ Pipeline implemented (2-stage or more)
- ☐ Peripheral added (UART, GPIO, or timer)

Appendix: Glossary

This glossary defines common terms used across the tutorial workbook.

A

Address Space

The range of memory addresses a CPU can access.

ALU (Arithmetic Logic Unit)

The hardware block that performs math and logic operations such as add, subtract, AND, and OR.

Assembler

A tool that converts assembly text into machine-readable binary or hex.

Asynchronous Read

A memory read where output changes immediately when the address changes (not waiting for a clock edge).

B

Backend (Compiler Backend)

The part of a compiler that turns intermediate code into instructions for a specific CPU architecture.

Bit Field

A group of bits inside an instruction used for one meaning, such as opcode, register index, or immediate.

Branch

An instruction that may change control flow by setting the PC to a different value.

Byte

8 bits of data.

Byte-Addressed Memory

A memory model where each address points to one byte.

C**Calling Convention**

Rules for how functions pass arguments, return values, and save registers.

Clock Edge

A transition of the clock signal (usually rising edge) where sequential logic updates state.

Combinational Logic

Logic whose outputs depend only on current inputs, not stored state.

Control Flow

The order in which instructions execute.

CPU (Central Processing Unit)

Hardware that fetches, decodes, and executes instructions.

D**Data Memory (DMEM)**

Memory used for runtime data (variables, arrays, stack), separate from instruction memory in Harvard-style designs.

Decoder

Hardware that interprets instruction bits and produces control signals.

F**Fetch**

Reading the next instruction from instruction memory.

FPGA (Field-Programmable Gate Array)

A reconfigurable chip used to implement digital hardware designs.

H**HALT**

An instruction or condition that stops program execution.

Harvard Architecture

An architecture with separate instruction memory and data memory paths.

Hex Image (.hex)

A text file containing machine code values used to initialize instruction memory.

I**Immediate**

A constant value encoded directly inside an instruction.

Instruction

A binary-encoded operation executed by the CPU.

Instruction Memory (IMEM)

Memory that stores program instructions.

ISA (Instruction Set Architecture)

The specification of a CPU's instruction formats, operations, registers, and behavior.

L**Link Register (LR)**

A register used to store return addresses for function calls.

Load

An instruction that reads data from memory into a register.

M**Machine Code**

Binary encoding of instructions executed directly by the CPU.

Module (Verilog Module)

A hardware building block with named ports and internal logic.

O**Opcode**

The instruction field that identifies which operation to perform.

Operand

Input data used by an instruction (register value, immediate, or memory location).

P**PC (Program Counter)**

A register holding the address (or index) of the current/next instruction.

pc_next

The computed next value of the program counter.

Pipeline

A CPU design technique where instruction processing is split into stages that overlap in time.

Plusargs

Runtime command-line parameters passed into Verilog simulation (for example, +PROGRAM=...).

R**Register**

A small, fast storage location inside the CPU.

Register File

The hardware array that stores general-purpose registers.

Reset

A signal that initializes CPU state to known values.

RTL (Register-Transfer Level)

A hardware design abstraction describing data movement between registers and combinational logic.

S**Sequential Logic**

Logic that updates state on clock edges.

Stack

A last-in-first-out memory region used for function calls, local data, and saved state.

Stack Pointer (SP)

A register pointing to the current top of the stack.

Store

An instruction that writes register data to memory.

Synthesis

Converting RTL code into a hardware netlist for FPGA or ASIC implementation.

SystemVerilog

An extension of Verilog with richer language features.

T**Testbench**

Simulation-only code that drives and observes a hardware design.

Trace

A step-by-step log of simulation behavior, often including PC and instruction values.

V**Verilog**

A hardware description language used to model digital systems.

VCD (Value Change Dump)

A waveform file format generated during simulation for viewing signal transitions.

W**Word**

A CPU's natural data unit size (commonly 16 or 32 bits in this project context).

Word-Addressed Memory

A memory model where each address points to one word, not one byte.

Workspace

The top-level project directory containing source files, scripts, and configs.